

Digital Design & Computer Arch.

Lecture 8: Instruction Set Architectures II

Prof. Onur Mutlu

ETH Zürich

Spring 2026

13 March 2026

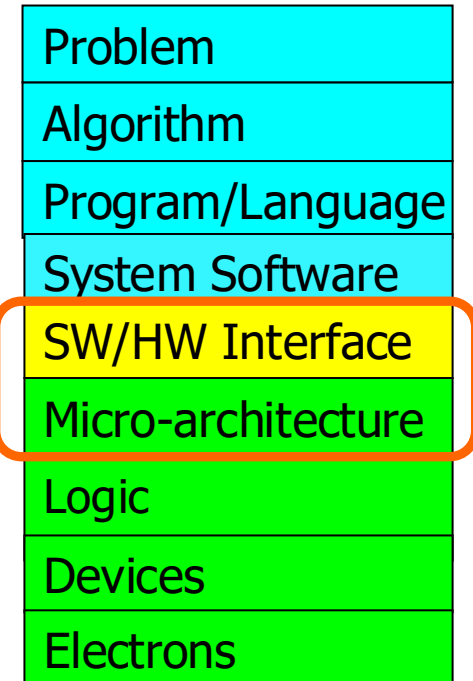
Agenda for Today & Next Few Lectures

- The von Neumann model
- LC-3: An example of von Neumann machine
- LC-3 and MIPS Instruction Set Architectures
- LC-3 and MIPS assembly and programming
- Introduction to microarchitecture and single-cycle microarchitecture
- Multi-cycle microarchitecture

Problem
Algorithm
Program/Language
System Software
SW/HW Interface
Micro-architecture
Logic
Devices
Electrons

What Will We Learn Today?

- Basic elements of a computer & the von Neumann model
 - LC-3: An example von Neumann machine
- Instruction Set Architectures: LC-3 and MIPS
 - Operate instructions
 - Data movement instructions
 - Control instructions
- Instruction formats
- Addressing modes



Readings

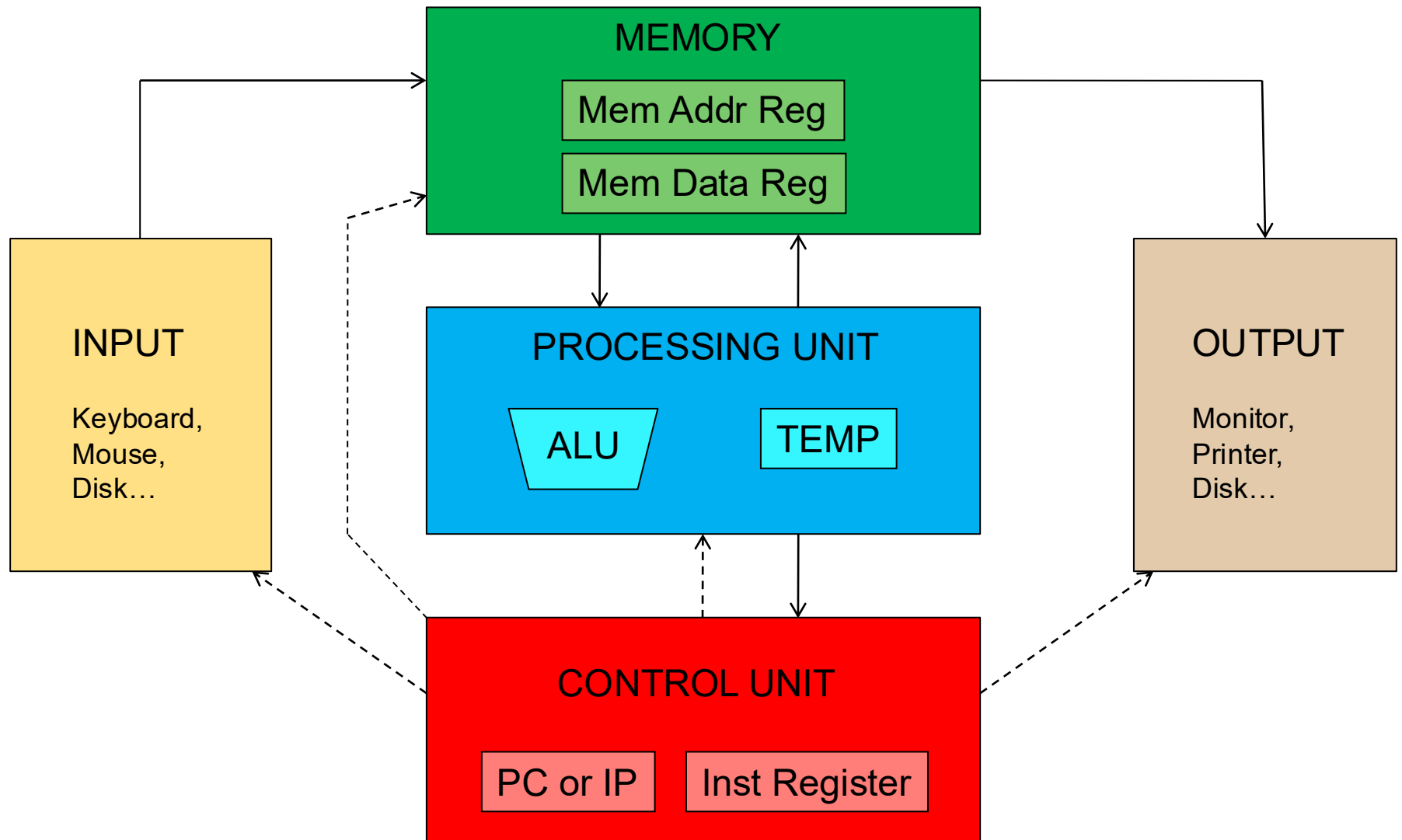
■ This week

- Von Neumann Model, ISA, LC-3, and MIPS
 - P&P, Chapters 4, 5 (we will follow these today & tomorrow)
 - H&H, Chapter 6 (until 6.5)
 - P&P, Appendices A and C (ISA and microarchitecture of LC-3)
 - H&H, Appendix B (MIPS instructions)
- Programming
 - P&P, Chapter 6 (we will follow this tomorrow)
- **Recommended:** H&H Chapter 5, especially 5.1, 5.2, 5.4, 5.5

■ Next week

- Introduction to microarchitecture and single-cycle microarchitecture
 - H&H, Chapter 7.1-7.3
 - P&P, Appendices A and C
- Multi-cycle microarchitecture
 - H&H, Chapter 7.4
 - P&P, Appendices A and C

Quick Recap: The von Neumann Model



Quick Recap: The von Neumann Model

Stored program

Sequential instruction processing

LC-3: A von Neumann Machine

LC-3: A von Neumann Machine

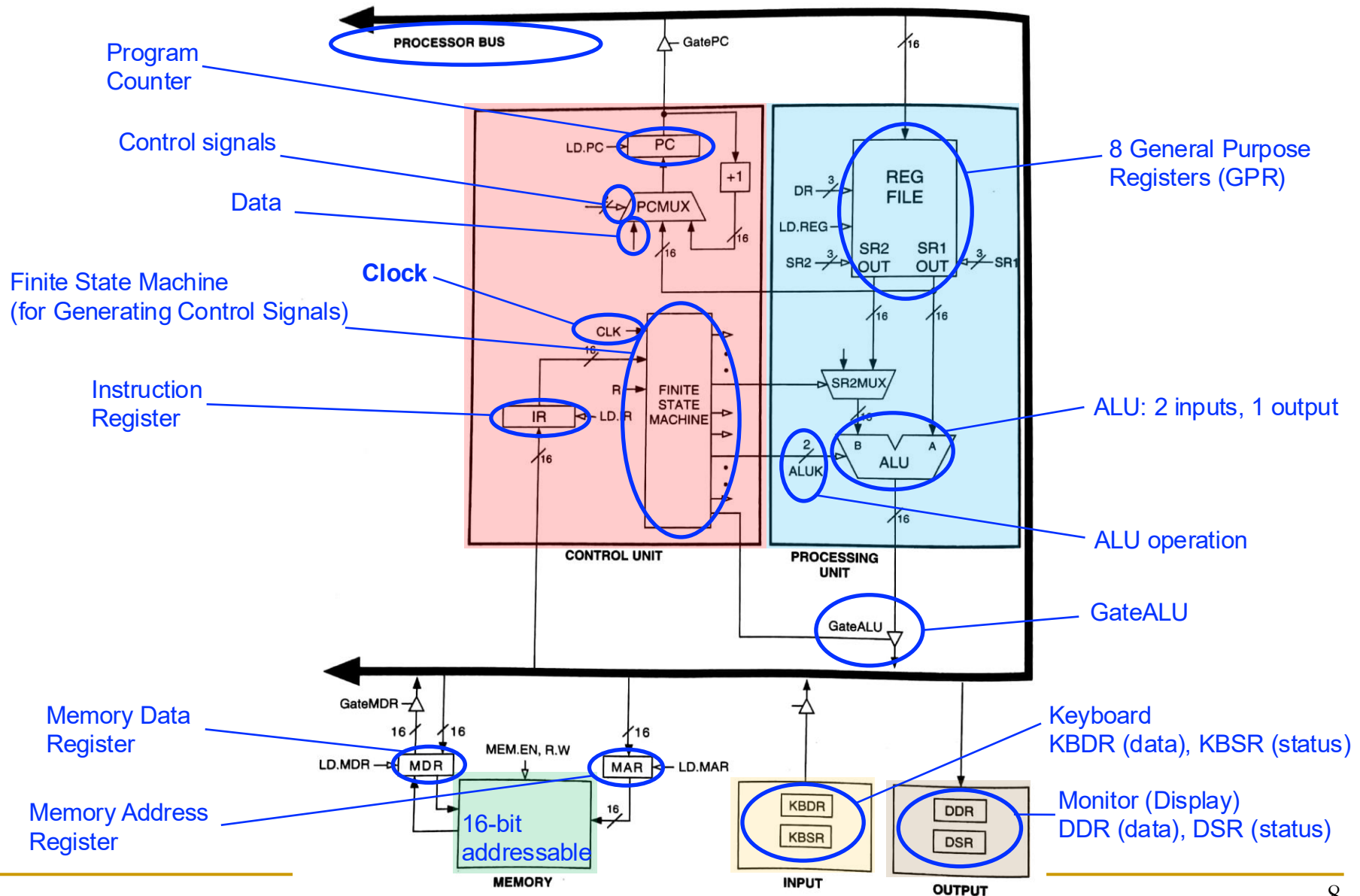


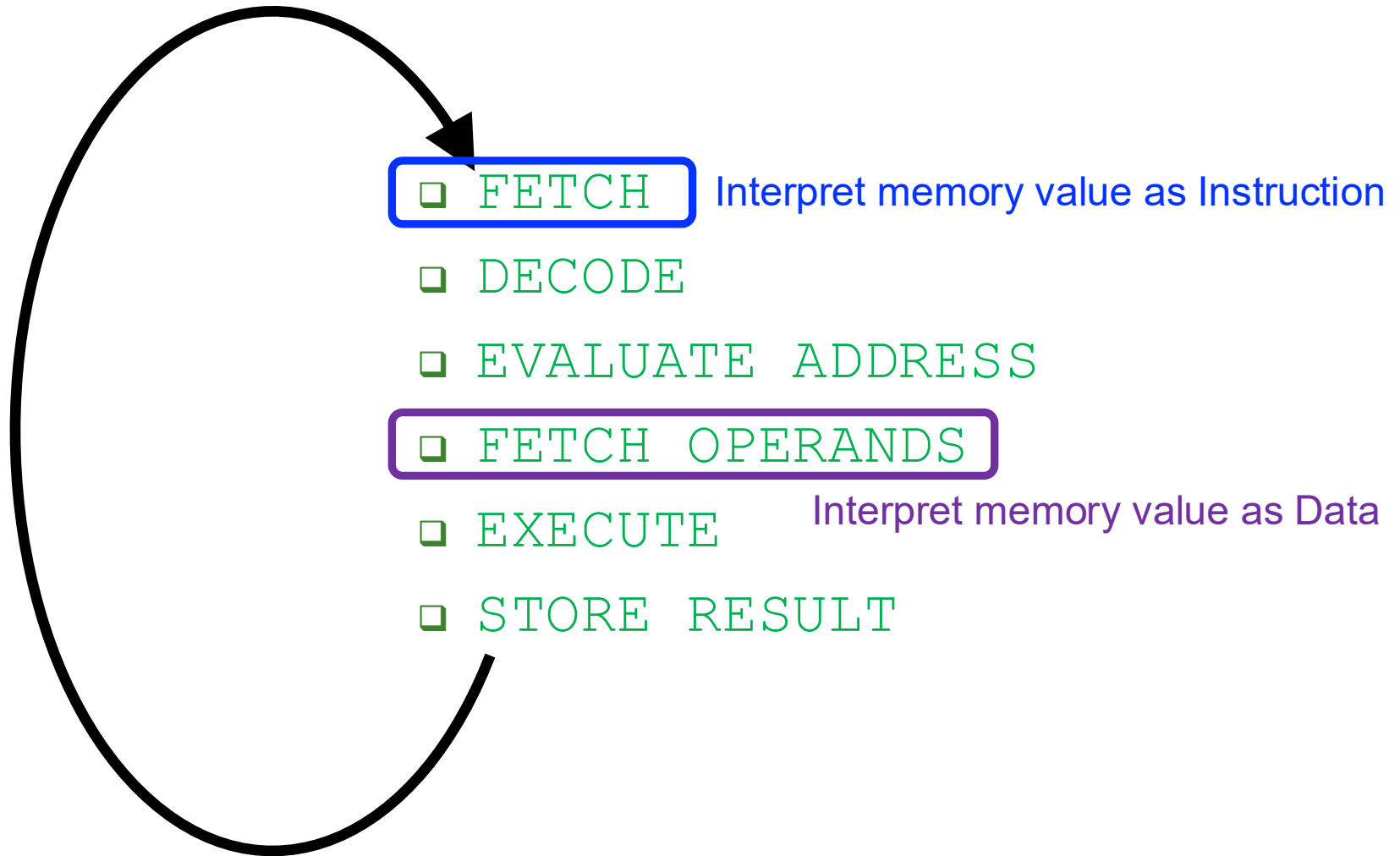
Figure 4.3 The LC-3 as an example of the von Neumann model

Review: Instruction Types

- There are three main types of instructions
- Operate instructions
 - Execute operations in the ALU
- Data movement instructions
 - Read from or write to memory
- Control flow instructions
 - Change the sequence of execution
- Let us start with some example instructions

Instruction (Processing) Cycle

Recall: The Instruction Cycle



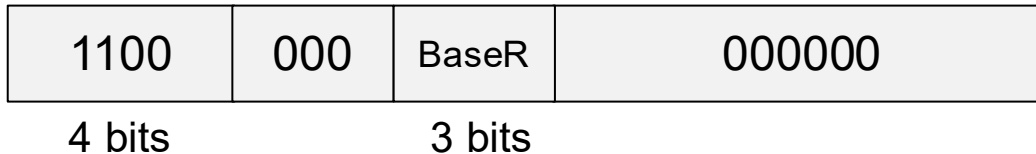
Whether a value fetched from memory is interpreted as an instruction depends on **when** that value is **fetched** in the instruction processing cycle

Recall: Jump in LC-3

- Unconditional branch or jump

- LC-3

JMP R2



- BaseR = Base register
- $PC \leftarrow R2$ (Register identified by BaseR)
- Variations
 - RET: special case of JMP where BaseR = R7
 - JSR, JSRR: jump to subroutine

This is **register**
addressing mode

PC UPDATE in LC-3

JMP loads
SR1 into PC

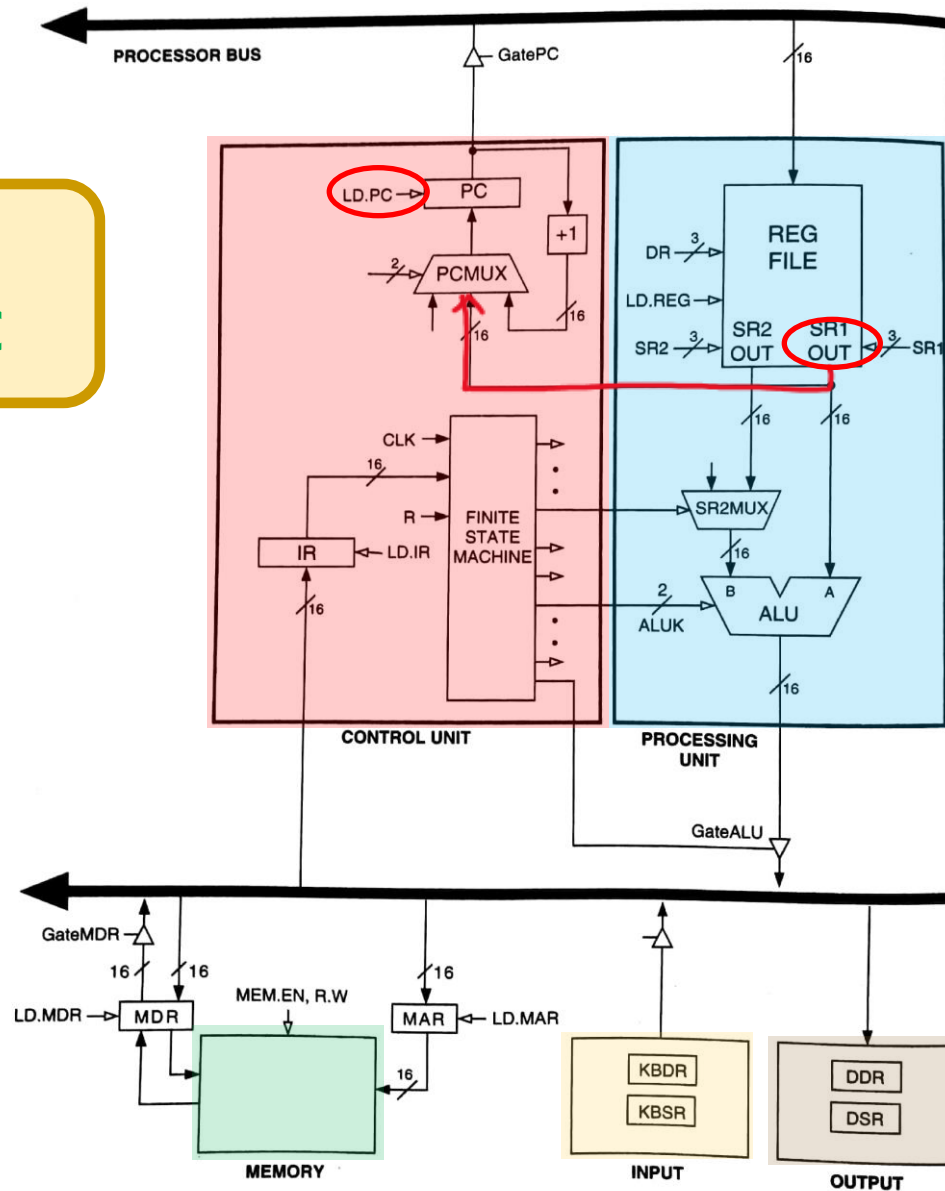
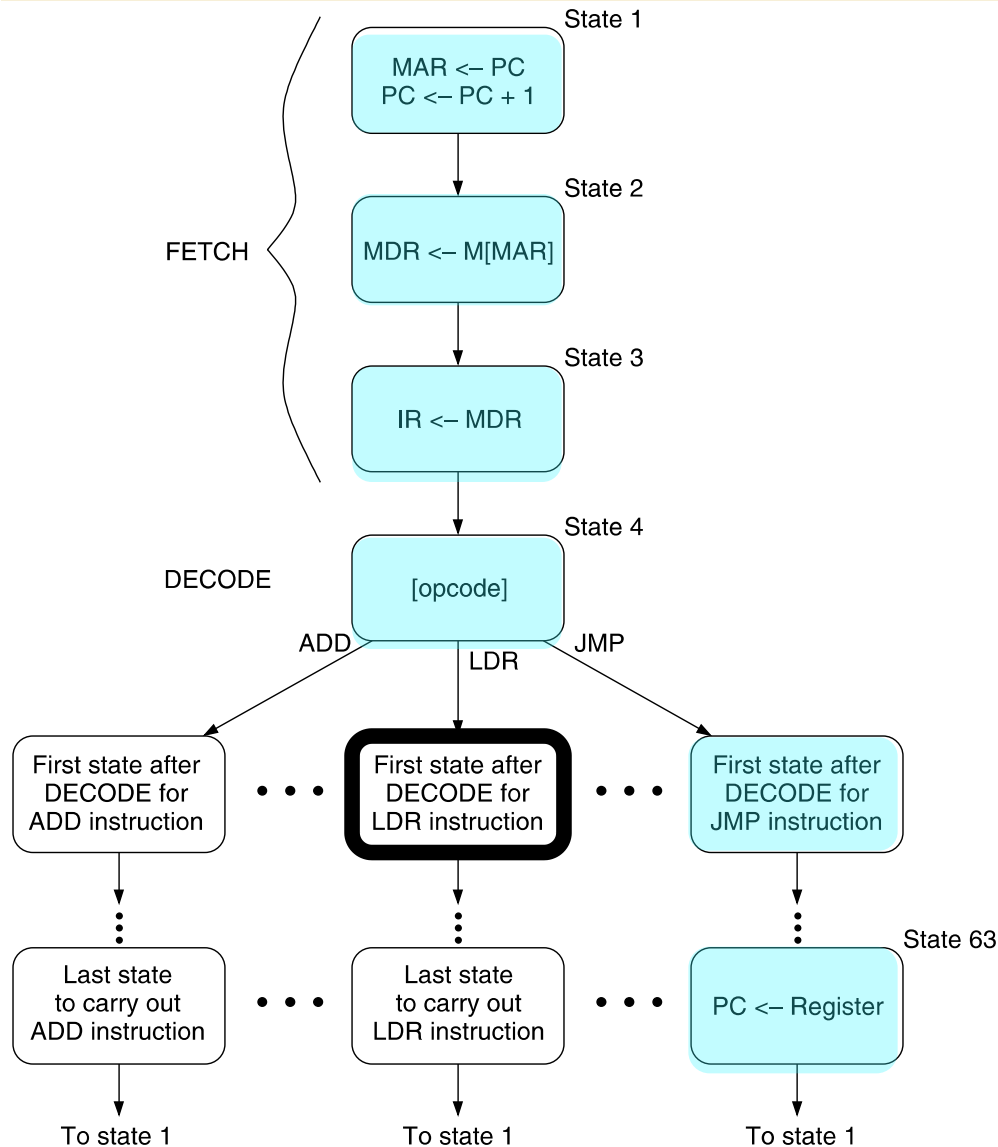


Figure 4.3 The LC-3 as an example of the von Neumann model

Control of the Instruction Cycle



- State 1
 - The FSM asserts GatePC and LD.MAR
 - It selects input (+1) in PCMUX and asserts LD.PC
- State 2
 - MDR is loaded with the instruction
- State 3
 - The FSM asserts GateMDR and LD.IR
- State 4
 - The FSM goes to next state depending on opcode
- State 63
 - JMP loads register into PC
- Full state diagram in Patt&Pattel, Appendix C

Figure 4.4 An abbreviated state diagram of the LC-3

To Come: Full State Machine for LC-3b

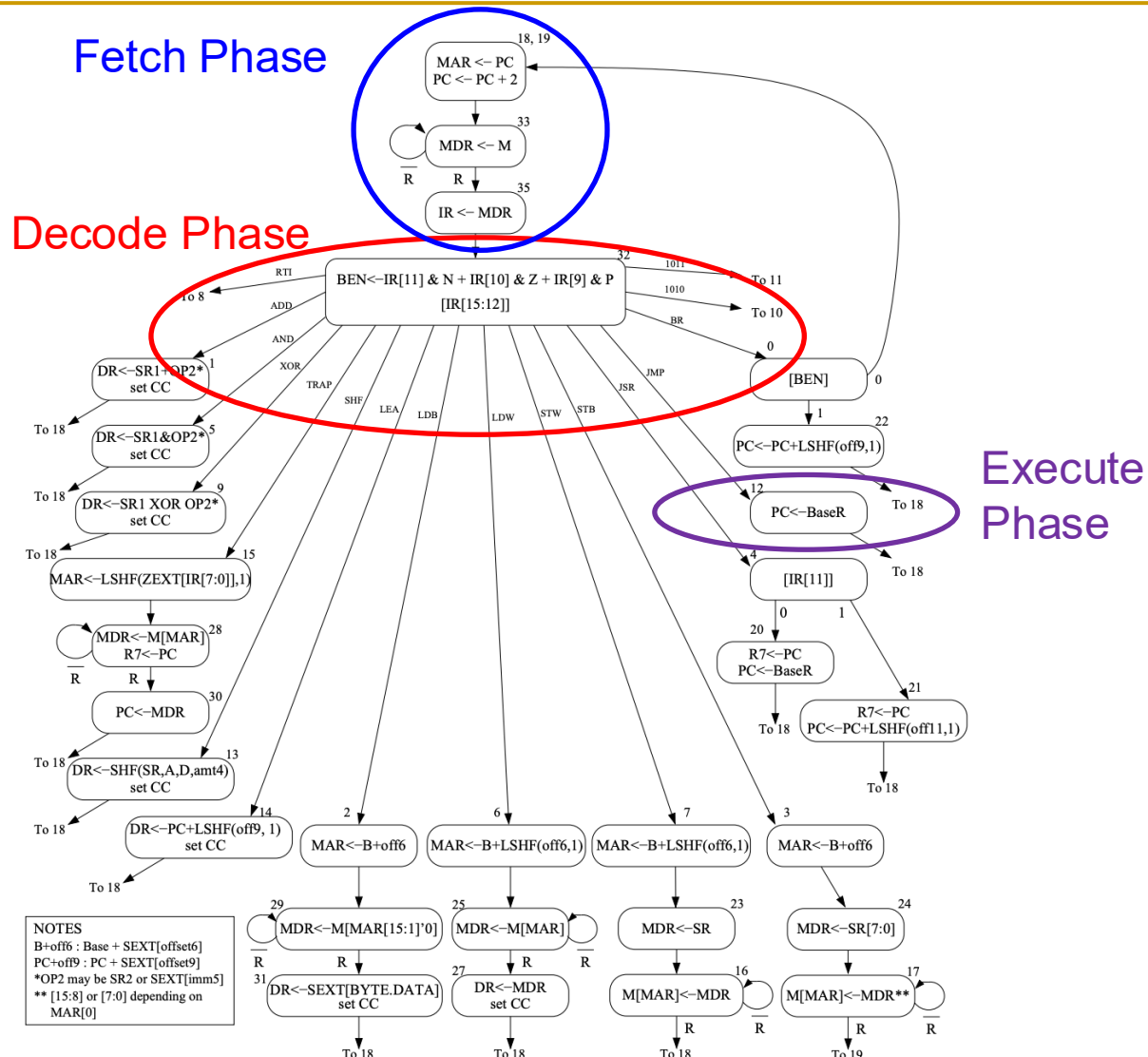
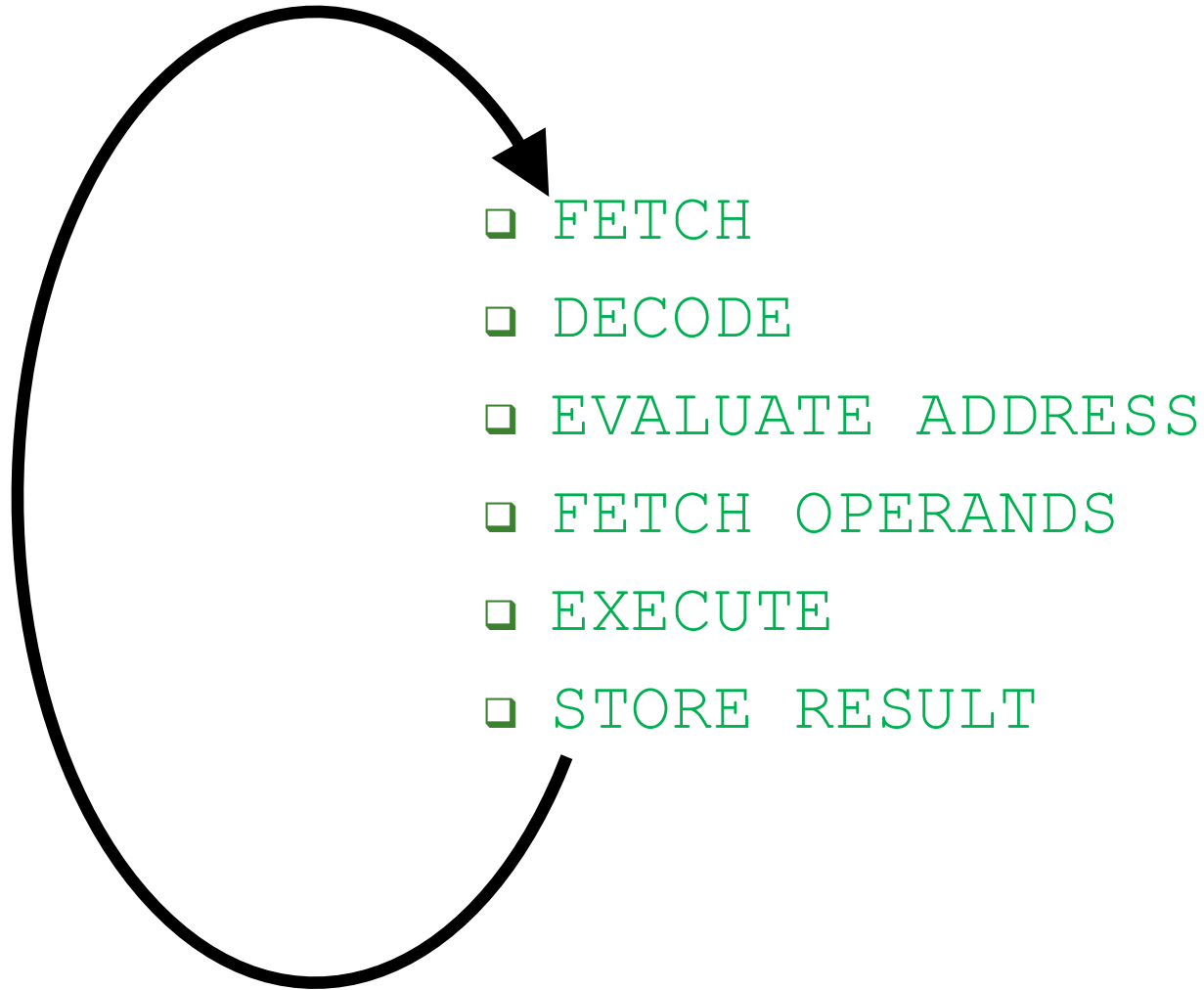


Figure C.2: A state machine for the LC-3b

The Instruction Cycle



LC-3 and MIPS

Instruction Set Architectures

Agenda for Today & Next Few Lectures

- The von Neumann model
- LC-3: An example of von Neumann machine
- LC-3 and MIPS Instruction Set Architectures
- LC-3 and MIPS assembly and programming
- Introduction to microarchitecture and single-cycle microarchitecture
- Multi-cycle microarchitecture

Problem
Algorithm
Program/Language
System Software
SW/HW Interface
Micro-architecture
Logic
Devices
Electrons

The Instruction Set

- It defines **opcodes**, **data types**, and **addressing modes**
- ADD and LDR have been our first examples

ADD

OP	DR	SR1	SR2		
1	0	1	0	00	2

Register mode

LDR

OP	DR	BaseR	offset6
6	3	0	4

Base+offset mode

The Instruction Set Architecture

- The ISA is the **interface between** what the **software** commands and what the **hardware** carries out
- The ISA specifies
 - The **memory organization**
 - Address space (LC-3: 2^{16} , MIPS: 2^{32})
 - Addressability (LC-3: 16 bits, MIPS: 8 bits)
 - Word- or Byte-addressable
 - The **register set**
 - 8 registers (R0 to R7) in LC-3
 - 32 registers in MIPS
 - The **instruction set**
 - **Opcodes**
 - **Data types**
 - **Addressing modes**
 - Length and format of instructions

Problem
Algorithm
Program
ISA
Microarchitecture
Circuits
Electrons

Instructions (Opcodes)

Opcodes

- A large or small **set of opcodes** could be defined
 - E.g, HP Precision Architecture: an instruction for $A*B+C$
 - E.g, x86 ISA: multimedia extensions (MMX), later SSE, AVX, AMX
 - E.g, VAX ISA: opcode to save all information of one program prior to switching to another program
- **Tradeoffs** are involved. Examples:
 - Hardware complexity vs. software complexity
 - Latency of simple vs. complex instructions
- In LC-3 and in MIPS there are three **types of opcodes**
 - Operate
 - Data movement
 - Control

Opcodes in LC-3

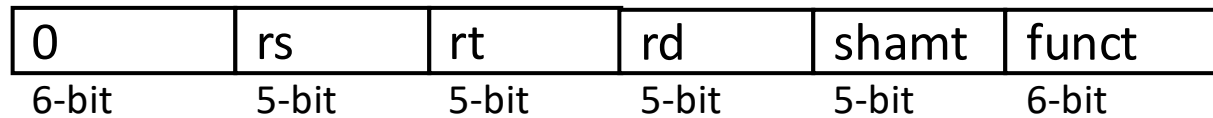
	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			0	00		SR2		
ADD ⁺	0001				DR			SR1			1	imm5				
AND ⁺	0101				DR			SR1			0	00		SR2		
AND ⁺	0101				DR			SR1			1	imm5				
BR	0000				n	z	p	PCOffset9								
JMP	1100				000			BaseR			000000					
JSR	0100				1	PCOffset11										
JSRR	0100				0	00		BaseR			000000					
LD ⁺	0010				DR			PCOffset9								
LDI ⁺	1010				DR			PCOffset9								
LDR ⁺	0110				DR			BaseR			offset6					
LEA ⁺	1110				DR			PCOffset9								
NOT ⁺	1001				DR			SR			111111					
RET	1100				000			111			000000					
RTI	1000				000000000000											
ST	0011				SR			PCOffset9								
STI	1011				SR			PCOffset9								
STR	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
reserved	1101															

Figure 5.3 Formats of the entire LC-3 instruction set. NOTE: ⁺ indicates instructions that modify condition codes

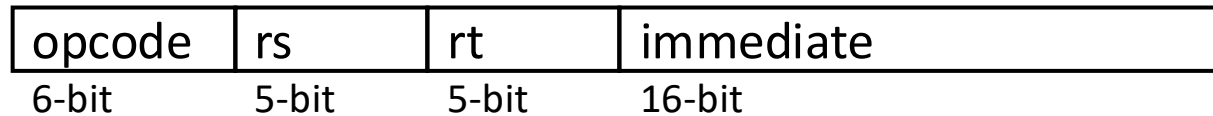
Opcodes in LC-3b

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			A	op.spec				
AND ⁺	0101				DR			SR1			A	op.spec				
BR	0000				n	z	p	PCOffset9								
JMP	1100				000			BaseR			000000					
JSR(R)	0100				A	operand.specifier										
LDB ⁺	0010				DR			BaseR			boffset6					
LDW ⁺	0110				DR			BaseR			offset6					
LEA ⁺	1110				DR			PCOffset9								
RTI	1000				000000000000											
SHF ⁺	1101				DR			SR			A	D	amount4			
STB	0011				SR			BaseR			boffset6					
STW	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
XOR ⁺	1001				DR			SR1			A	op.spec				
not used	1010															
not used	1011															

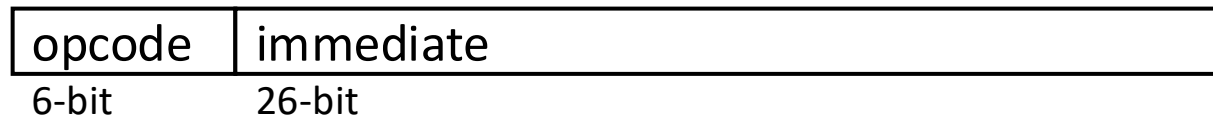
MIPS Instruction Types



R-type



I-type



J-type

Funct in MIPS R-Type Instructions (I)

Opcode is 0
in MIPS
R-Type
instructions.

Funct defines
the operation

Table B.2 R-type instructions, sorted by funct field

Funct	Name	Description	Operation
000000 (0)	sll rd, rt, shamt	shift left logical	[rd] = [rt] << shamt
000010 (2)	srl rd, rt, shamt	shift right logical	[rd] = [rt] >> shamt
000011 (3)	sra rd, rt, shamt	shift right arithmetic	[rd] = [rt] >>> shamt
000100 (4)	sllv rd, rt, rs	shift left logical variable	[rd] = [rt] << [rs] _{4:0}
000110 (6)	srlv rd, rt, rs	shift right logical variable	[rd] = [rt] >> [rs] _{4:0}
000111 (7)	srav rd, rt, rs	shift right arithmetic variable	[rd] = [rt] >>> [rs] _{4:0}
001000 (8)	jr rs	jump register	PC = [rs]
001001 (9)	jalr rs	jump and link register	\$ra = PC + 4, PC = [rs]
001100 (12)	syscall	system call	system call exception
001101 (13)	break	break	break exception
010000 (16)	mfhi rd	move from hi	[rd] = [hi]
010001 (17)	mti rs	move to hi	[hi] = [rs]
010010 (18)	mflo rd	move from lo	[rd] = [lo]
010011 (19)	mtlo rs	move to lo	[lo] = [rs]
011000 (24)	mult rs, rt	multiply	{[hi], [lo]} = [rs] × [rt]
011001 (25)	multu rs, rt	multiply unsigned	{[hi], [lo]} = [rs] × [rt]
011010 (26)	div rs, rt	divide	[lo] = [rs]/[rt], [hi] = [rs]%[rt]
011011 (27)	divu rs, rt	divide unsigned	[lo] = [rs]/[rt], [hi] = [rs]%[rt]

(continued)

Funct in MIPS R-Type Instructions (II)

Table B.2 R-type instructions, sorted by funct field—Cont'd

Funct	Name	Description	Operation
100000 (32)	add rd, rs, rt	add	[rd] = [rs] + [rt]
100001 (33)	addu rd, rs, rt	add unsigned	[rd] = [rs] + [rt]
100010 (34)	sub rd, rs, rt	subtract	[rd] = [rs] - [rt]
100011 (35)	subu rd, rs, rt	subtract unsigned	[rd] = [rs] - [rt]
100100 (36)	and rd, rs, rt	and	[rd] = [rs] & [rt]
100101 (37)	or rd, rs, rt	or	[rd] = [rs] [rt]
100110 (38)	xor rd, rs, rt	xor	[rd] = [rs] ^ [rt]
100111 (39)	nor rd, rs, rt	nor	[rd] = ~([rs] [rt])
101010 (42)	slt rd, rs, rt	set less than	[rs] < [rt] ? [rd] = 1 : [rd] = 0
101011 (43)	sltu rd, rs, rt	set less than unsigned	[rs] < [rt] ? [rd] = 1 : [rd] = 0

- More complete list of instructions are in H&H Appendix B

Data Types

Data Types

- An ISA supports one or several data types
- LC-3 only supports 2's complement integers **H&H Chapter 1.4.6**
 - Negative of a 2's complement binary value $X = \text{NOT}(X) + 1$
- MIPS supports
 - 2's complement integers
 - Unsigned integers
 - Floating point
- **Tradeoffs** are involved. Examples:
 - Hardware complexity vs. software complexity
 - Latency of operations on supported vs. unsupported data types

Why Have Different Data Types in ISA?

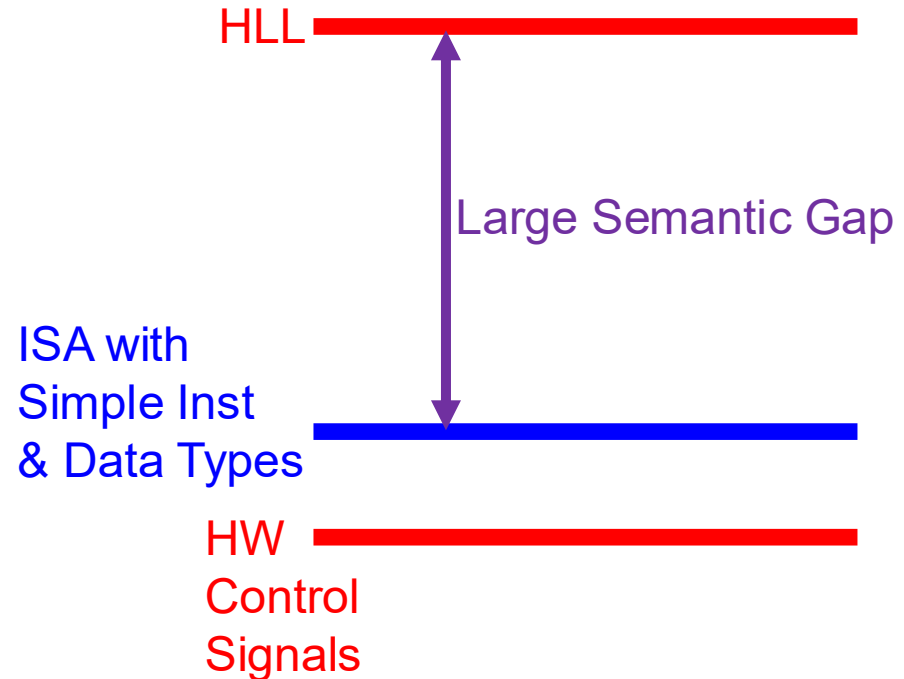
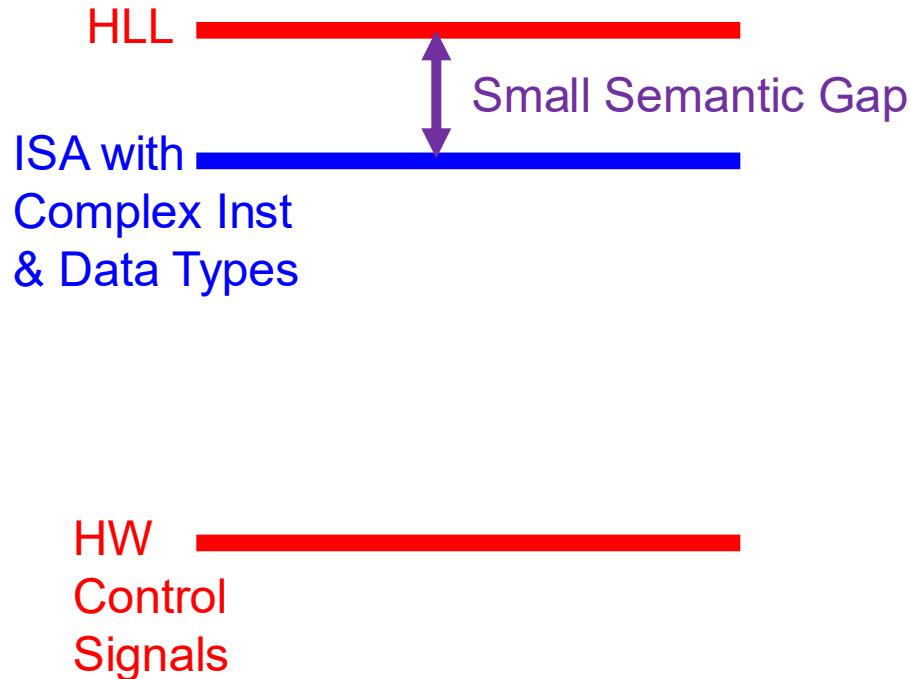
- An example of programmer vs. microarchitect tradeoff
- Advantage of more data types:
 - Enables better mapping of high-level programming constructs to hardware
 - Hardware can directly operate on data types present in programming languages → small number of instructions and code size
 - Matrix operations vs. individual multiply/add/load/store instructions
 - Graph operations vs. individual load/store/add/... instructions
- Disadvantage:
 - More work for the microarchitect
 - who needs to implement the data types and instructions that operate on data types

Data Types and Instruction Complexity

- Data types are coupled tightly to the semantic level, or **complexity of instructions**
- Concept of **semantic gap**
 - how close instructions & data types are to high-level language
- Complex instructions + data types → small semantic gap
 - E.g., insert into a doubly linked list, multiply two matrices
 - VAX ISA: doubly-linked list, multi-dimensional arrays
- Simple instructions + data types → large semantic gap
 - E.g., primitive operations: load, store, multiply, add, nor
 - Early RISC machines: Only integer data type, simple operations

Semantic Gap

- How close instructions & data types are to high-level language (HLL)



Complex vs. Simple Instructions+Data Types

- **Complex instruction**: An instruction **does a lot of work**, e.g. many operations
 - ❑ Insert in a doubly linked list
 - ❑ Compute FFT
 - ❑ String copy
 - ❑ Matrix multiply
 - ❑ ...
- **Simple instruction**: An instruction **does little work** -- it is a primitive using which complex operations can be built
 - ❑ Add
 - ❑ XOR
 - ❑ Multiply
 - ❑ ...

Complex vs. Simple Instructions+Data Types

■ Advantages of Complex Instructions + Data Types

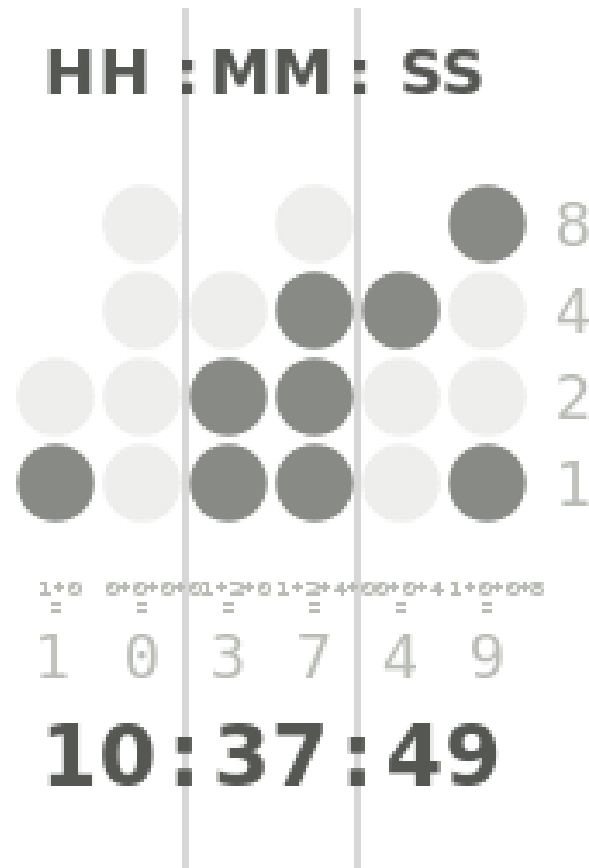
- + **Denser encoding** → smaller code size → better memory utilization, saves off-chip bandwidth, better cache hit rate (better packing of instructions)
- + **Simpler compiler**: no need to optimize small instructions as much

■ Disadvantages of Complex Instructions + Data Types

- **Larger chunks of work** → compiler has less opportunity to optimize (limited in fine-grained optimizations it can do)
- **More complex hardware** → translation from a high level to control signals and optimization needs to be done by hardware

Aside: An Example: **Binary**Coded**D**ecimal

- Each decimal digit is encoded with a fixed number of bits



Aside: An Example: **Binary**Coded**D**ecimal

- Each decimal digit is encoded with a fixed number of bits



"Binary clock" by Alexander Jones & Eric Pierce - Own work, based on Wapcaplet's Binary clock.png on the English Wikipedia. Licensed under CC BY-SA 3.0 via Wikimedia Commons - http://commons.wikimedia.org/wiki/File:Binary_clock.svg#mediaviewer/File:Binary_clock.svg

"Digital-BCD-clock" by Julo - Own work. Licensed under Public Domain via Wikimedia Commons - <http://commons.wikimedia.org/wiki/File:Digital-BCD-clock.jpg#mediaviewer/File:Digital-BCD-clock.jpg>

Addressing Modes

Addressing Modes

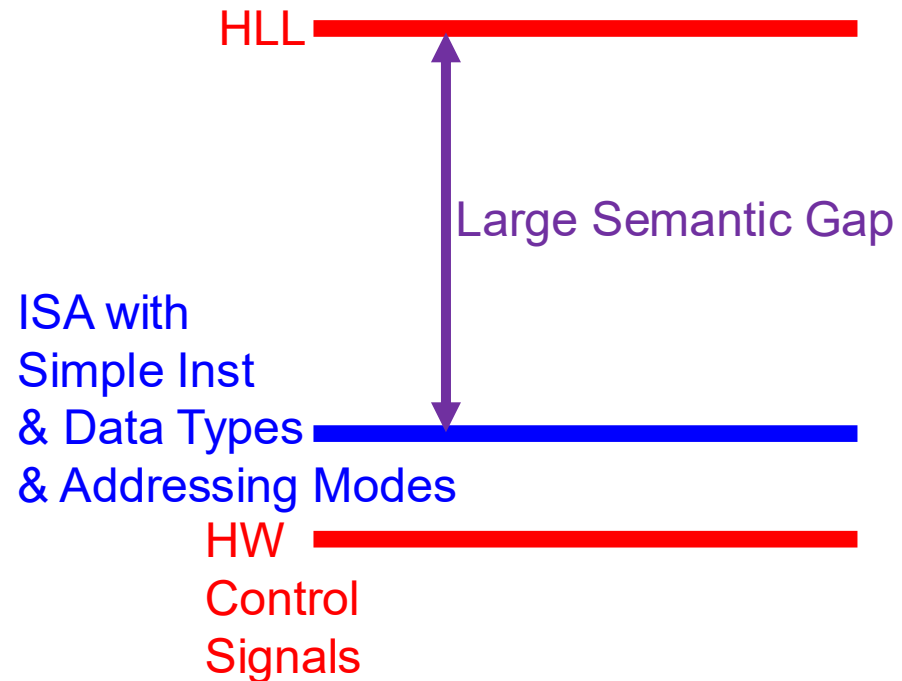
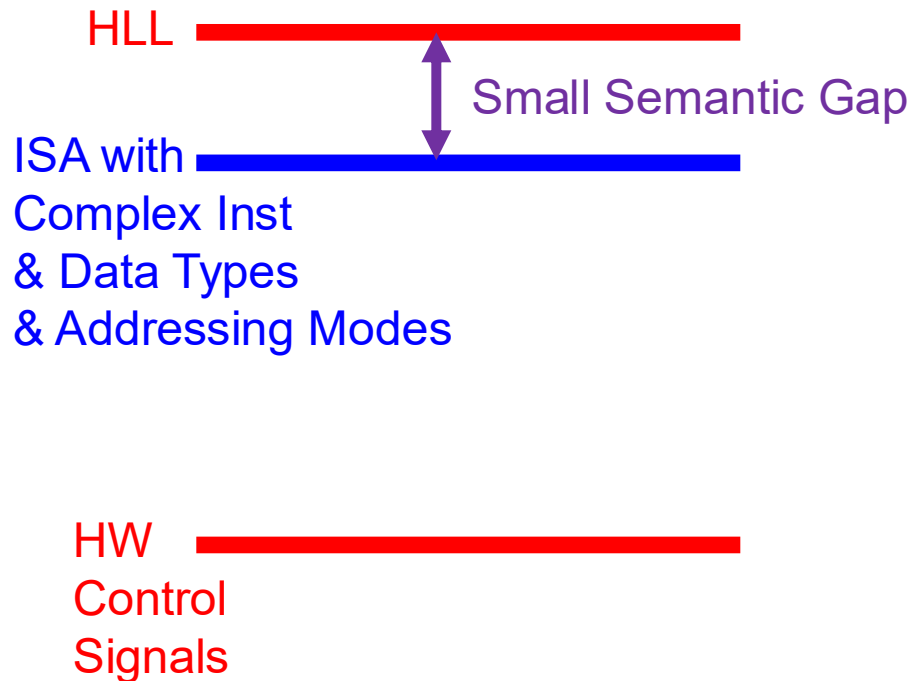
- An addressing mode is a mechanism for specifying where an operand is located
- There are five addressing modes in LC-3
 - Immediate or literal (constant)
 - The operand is in some bits of the instruction
 - Register
 - The operand is in one of R0 to R7 registers
 - Three memory addressing modes
 - PC-relative
 - Indirect
 - Base+offset
- MIPS has pseudo-direct addressing (for j and jal), additionally, but does **not** have indirect addressing

Why Have Different Addressing Modes?

- Another example of programmer vs. microarchitect tradeoff
- **Advantage of more addressing modes:**
 - Enables better mapping of high-level programming constructs to hardware
 - some accesses are better expressed with a different mode → reduced number of instructions and code size
 - Array indexing (one or multi-dimensional)
 - Pointer-based accesses (indirection)
 - Matrix element indexing; sparse matrix element indexing
- **Disadvantages:**
 - More work for the microarchitect
 - More options for the compiler to decide what to use

Semantic Gap Applies to Addressing Modes

- How close instructions & data types & addressing modes are to high-level language (HLL)



Many Tradeoffs in ISA Design...

- Execution model – sequencing model and processing style
- Instruction length
- Instruction format
- Instruction types
- Instruction complexity vs. simplicity
- Data types
- Number of registers
- Addressing mode types
- Memory organization (address space, addressability, endianness, ...)
- Memory access restrictions and permissions
- Support for multiple instructions to execute in parallel?
- ...

Operate Instructions

Operate Instructions

- In **LC-3**, there are three operate instructions
 - NOT is a **unary operation** (one source operand)
 - It executes bitwise NOT
 - ADD and AND are **binary operations** (two source operands)
 - ADD is 2's complement addition
 - AND is bitwise SR1 & SR2
- In **MIPS**, there are many more
 - Many **R-type** instructions (they are **binary operations**)
 - E.g., add, and, nor, xor...
 - **I-type** versions (i.e., with one immediate operand) of the R-type operate instructions
 - **F-type** operations, i.e., floating-point operations

NOT in LC-3

■ NOT assembly and machine code

LC-3 assembly

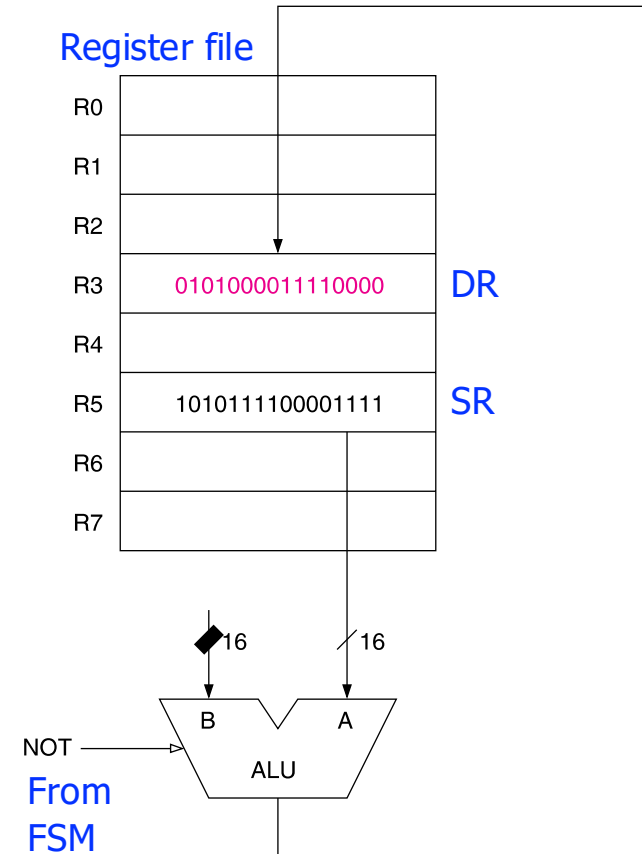
```
NOT R3, R5
```

Field Values

OP	DR	SR	
9	3	5	1 1 1 1 1 1

Machine Code

OP	DR	SR	
1 0 0 1	0 1 1	0 0 1	1 1 1 1 1 1
15	12	11 9	8 6 5 0



There is **no NOT in MIPS**. How is it implemented?

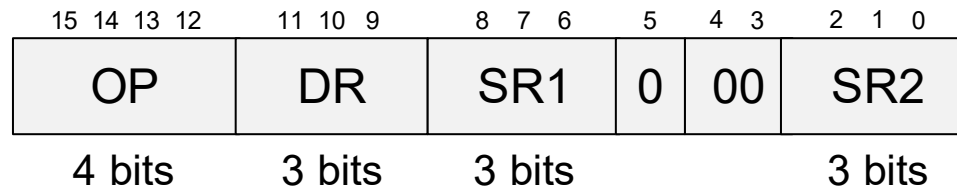
There is **no NOT in LC-3b**. How is it implemented?

Operate Instructions

- We are already familiar with LC-3's ADD and AND with register mode (R-type in MIPS)
- Now let us see the versions with one literal (i.e., immediate) operand
- We will use Subtraction as an example
 - How is it implemented in LC-3 and MIPS?

Recall: LC-3 Operate Instruction Format

■ LC-3 Operate Instruction Format (Register OP Register)



□ OP = **opcode** (what the instruction does)

■ E.g., ADD = 0001

□ **Semantics:** $DR \leftarrow SR1 + SR2$

■ E.g., AND = 0101

□ **Semantics:** $DR \leftarrow SR1 \text{ AND } SR2$

□ SR1, SR2 = source registers

□ DR = destination register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	0	0	1	0	0	0	0	1	1	0
ADD				R6			R2			R6					

Operate Instr. with one Literal in LC-3

■ ADD and AND



- OP = operation
 - E.g., **ADD** = 0001 (same OP as the register-mode ADD)
 - $DR \leftarrow SR1 + \text{sign-extend}(\text{imm5})$
 - E.g., **AND** = 0101 (same OP as the register-mode AND)
 - $DR \leftarrow SR1 \text{ AND } \text{sign-extend}(\text{imm5})$
- SR1 = source register
- DR = destination register
- **imm5** = Literal or immediate (sign-extend to 16 bits)

ADD with one Literal in LC-3

■ ADD assembly and machine code

LC-3 assembly

```
ADD R1, R4, #-2
```

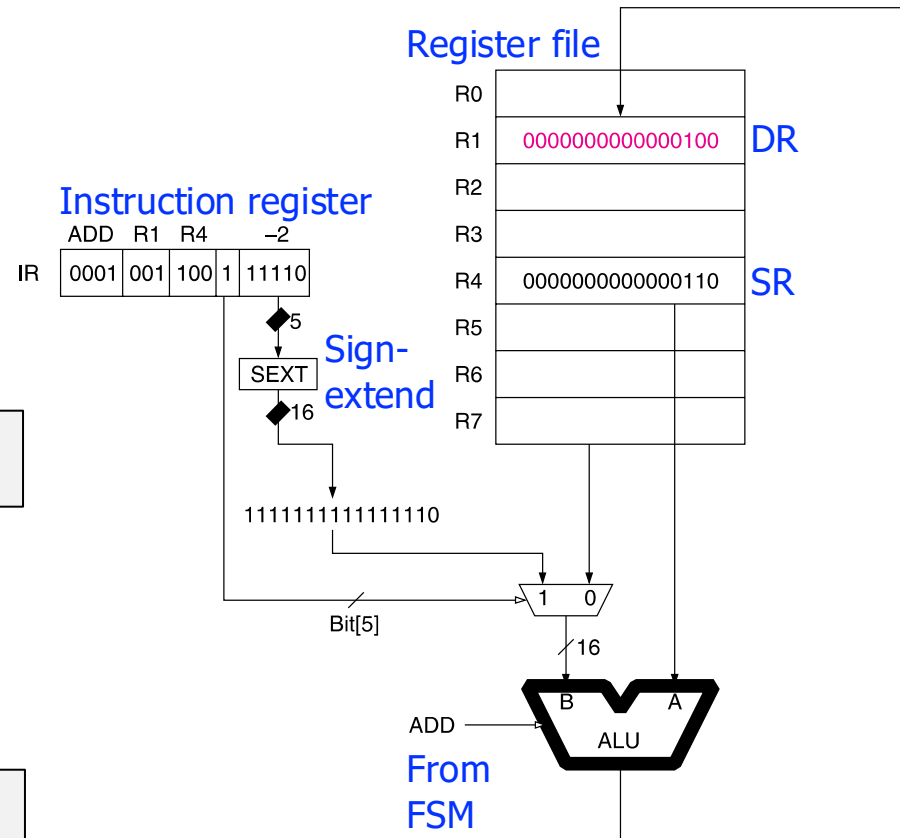
Field Values

OP	DR	SR		imm5
1	1	4	1	-2

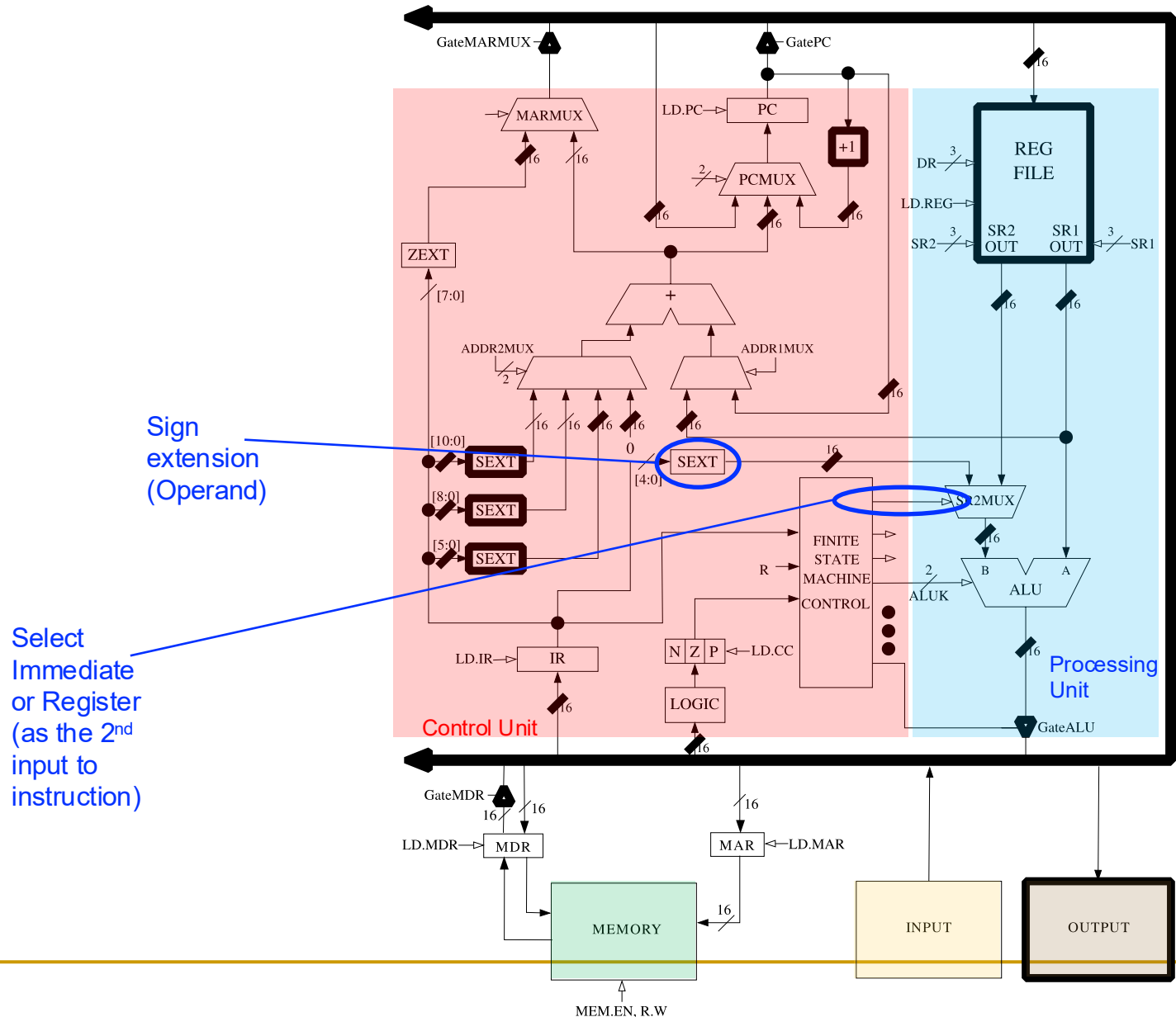
Machine Code

OP	DR	SR		imm5
0001	001	100	1	11110

15 12 11 9 8 6 5 4 0

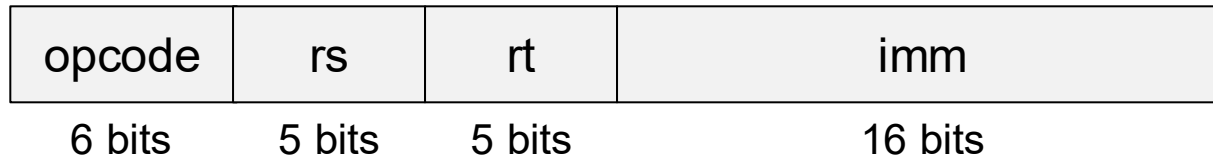


ADD with one Literal in LC-3 Data Path



Instructions with one Literal in MIPS

- I-type MIPS Instructions
 - 2 register operands and immediate
- Some operate and data movement instructions



- opcode = operation
- rs = source register
- rt =
 - destination register in some instructions (e.g., `addi`, `lw`)
 - source register in others (e.g., `sw`)
- imm = Literal or immediate

ADD with one Literal in MIPS

■ Add immediate

MIPS assembly

```
addi $s0, $s1, 5
```

Field Values

op	rs	rt	imm
8	17	16	5

$rt \leftarrow rs + \text{sign-extend}(\text{imm})$

Machine Code

op	rs	rt	imm
001000	10001	10010	0000 0000 0000 0101

0x22300005

Subtraction in MIPS vs. LC-3

■ MIPS assembly

High-level code

```
a = b + c - d;
```

MIPS assembly

```
add    $t0, $s0, $s1
sub     $s3, $t0, $s2
```

■ LC-3 assembly

High-level code

```
a = b + c - d;
```

LC-3 assembly

```
ADD    R2, R0, R1
NOT     R4, R3
ADD     R5, R4, #1
ADD     R6, R2, R5
```

2's complement of R3

■ Tradeoff in LC-3

- ❑ More instructions
- ❑ But, simpler control logic

Subtract Immediate

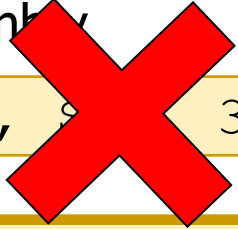
■ MIPS assembly

High-level code

```
a = b - 3;
```

MIPS assembly

```
subi $s1, $s0, 3
```



Is **subi** necessary in MIPS?

MIPS assembly

```
addi $s1, $s0, -3
```

■ LC-3

High-level code

```
a = b - 3;
```

LC-3 assembly

```
ADD R1, R0, #-3
```

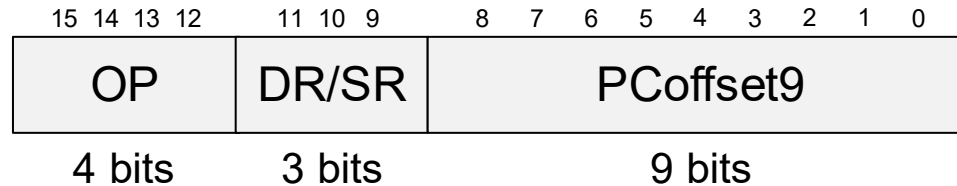
Data Movement Instructions and Addressing Modes

Data Movement Instructions

- In **LC-3**, there are seven data movement instructions
 - LD, LDR, LDI, LEA, ST, STR, STI
- Format of load and store instructions
 - Opcode (bits [15:12])
 - DR or SR (bits [11:9])
 - Address generation bits (bits [8:0])
 - Four ways to interpret bits, called **addressing modes**
 - PC-Relative Mode
 - Indirect Mode
 - Base+Offset Mode
 - Immediate Mode
- In **MIPS**, there are only **Base+offset** and **Immediate modes** for load and store instructions

PC-Relative Addressing Mode

■ LD (Load) and ST (Store)



- OP = opcode
 - E.g., LD = 0010
 - E.g., ST = 0011
- DR = destination register in LD
- SR = source register in ST
- LD: $DR \leftarrow \text{Memory}[PC^{\dagger} + \text{sign-extend}(\text{PCOffset9})]$
- ST: $\text{Memory}[PC^{\dagger} + \text{sign-extend}(\text{PCOffset9})] \leftarrow SR$

[†] This is the incremented PC

LD in LC-3

LD assembly and machine code

LC-3 assembly

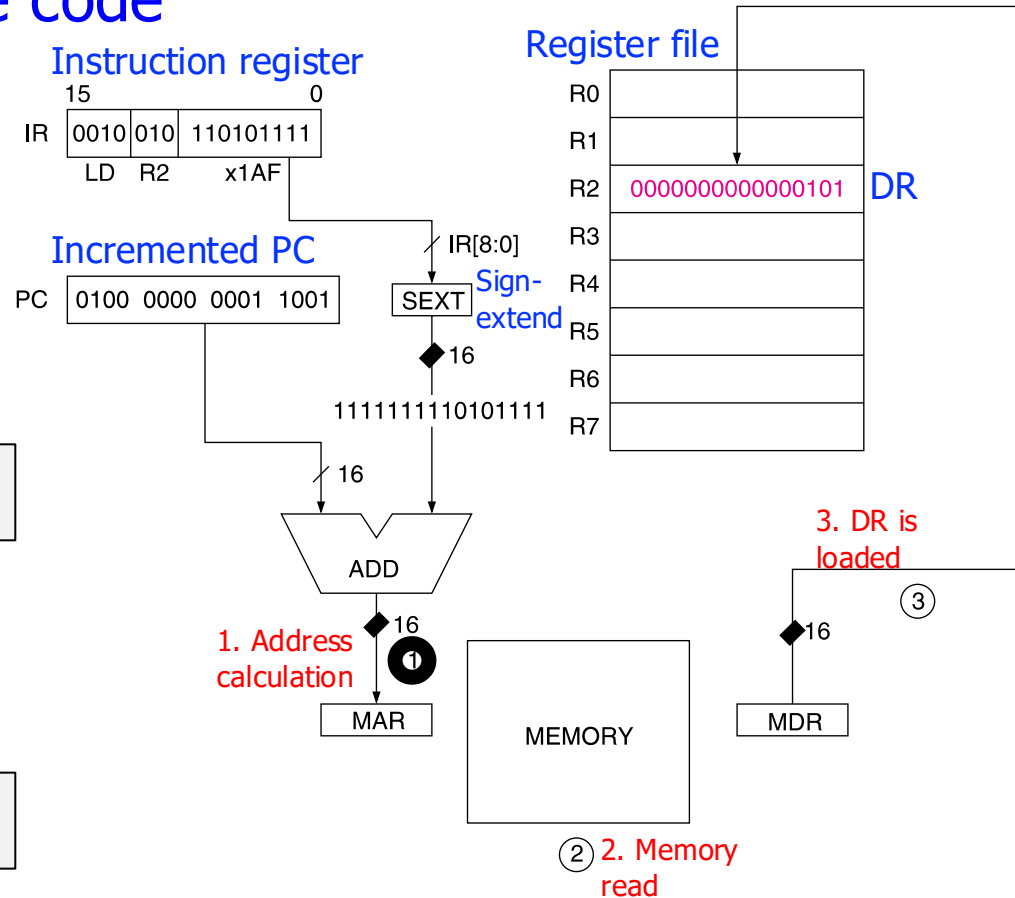
```
LD R2, 0x1AF
```

Field Values

OP	DR	PCOffset9
2	2	0x1AF

Machine Code

OP	DR	PCOffset9
0 0 1 0	0 1 0	1 1 0 1 0 1 1 1 1
15	12	11 9 8 0

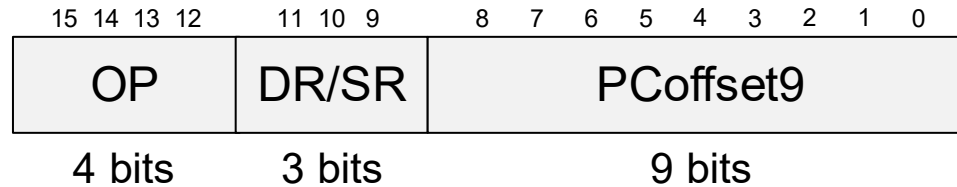


The memory address is **only +255 to -256** locations away of the **LD or ST instruction**

Limitation: The **PC-relative addressing mode** cannot address far away from the instruction

Indirect Addressing Mode

■ LDI (Load Indirect) and STI (Store Indirect)



- OP = opcode
 - E.g., LDI = 1010
 - E.g., STI = 1011
- DR = destination register in LDI
- SR = source register in STI
- LDI: $DR \leftarrow \text{Memory}[\text{Memory}[PC^{\dagger} + \text{sign-extend}(\text{PCOffset9})]]$
- STI: $\text{Memory}[\text{Memory}[PC^{\dagger} + \text{sign-extend}(\text{PCOffset9})]] \leftarrow SR$

[†] This is the incremented PC

LDI in LC-3

LDI assembly and machine code

LC-3 assembly

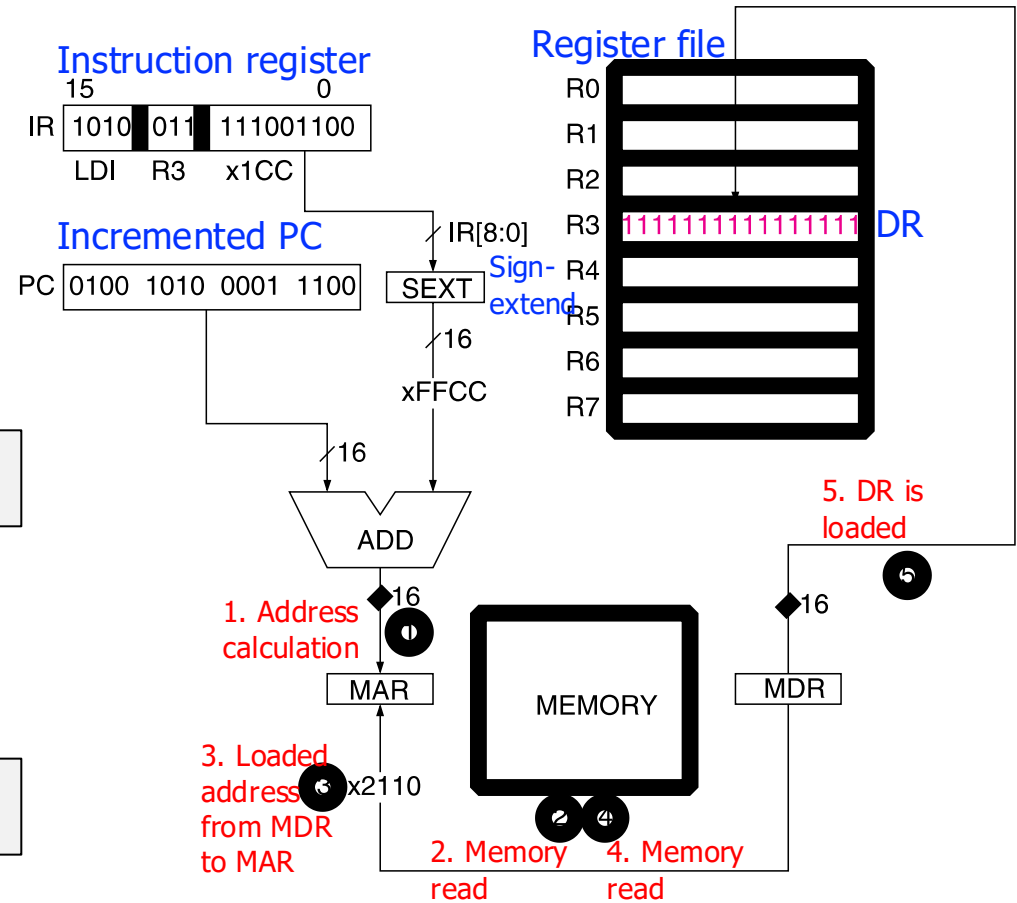
```
LDI R3, 0x1CC
```

Field Values

OP	DR	PCOffset9
A	3	0x1CC

Machine Code

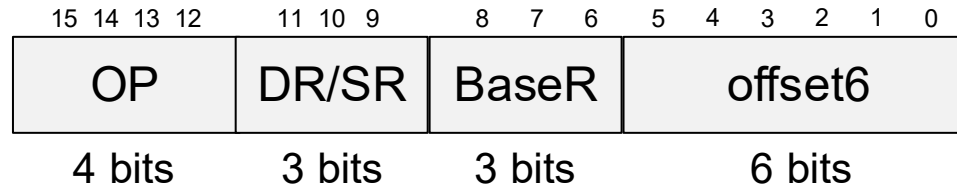
OP	DR	PCOffset9
1 0 1 0	0 1 1	1 1 1 0 0 1 1 0 0
15	12	11 9 8 0



Now the address of the operand can be anywhere in the memory

Base+Offset Addressing Mode

■ LDR (Load Register) and STR (Store Register)



- OP = opcode
 - E.g., LDR = 0110
 - E.g., STR = 0111
- DR = destination register in LDR
- SR = source register in STR
- LDR: $DR \leftarrow \text{Memory}[\text{BaseR} + \text{sign-extend}(\text{offset6})]$
- STR: $\text{Memory}[\text{BaseR} + \text{sign-extend}(\text{offset6})] \leftarrow SR$

LDR in LC-3

LDR assembly and machine code

LC-3 assembly

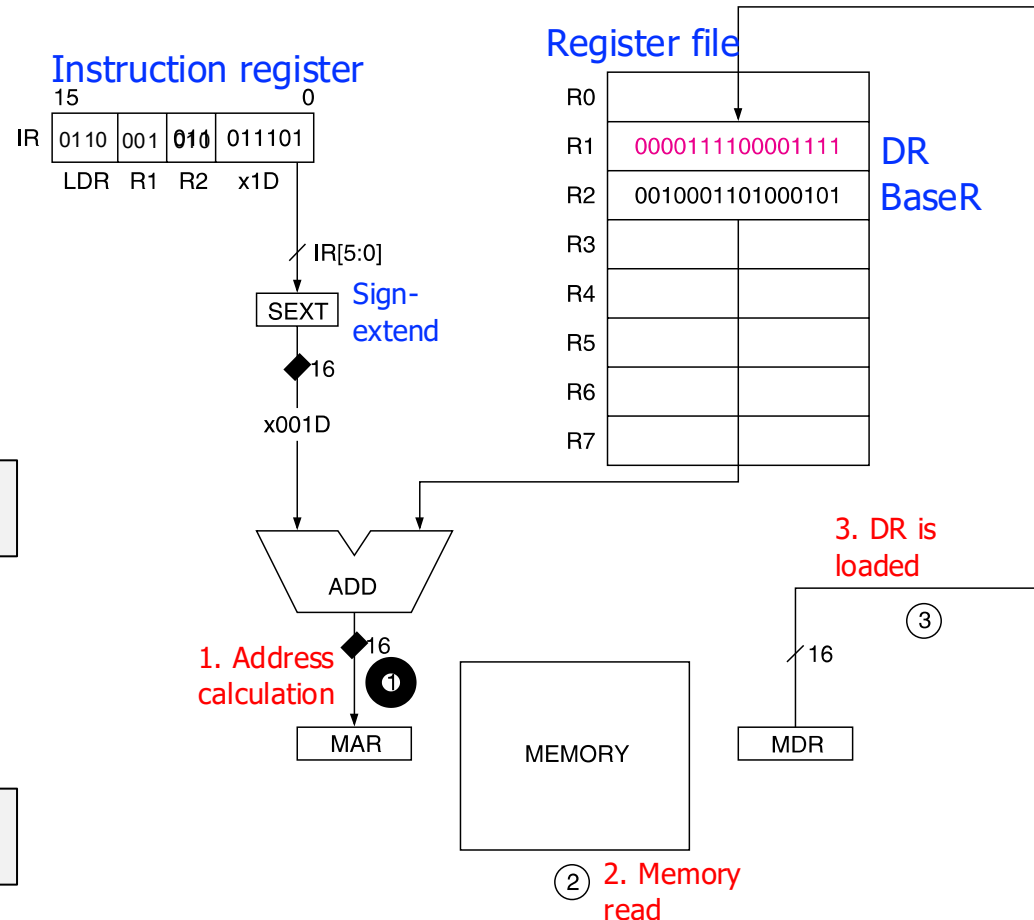
```
LDR R1, R2, 0x1D
```

Field Values

OP	DR	BaseR	offset6
6	1	2	0x1D

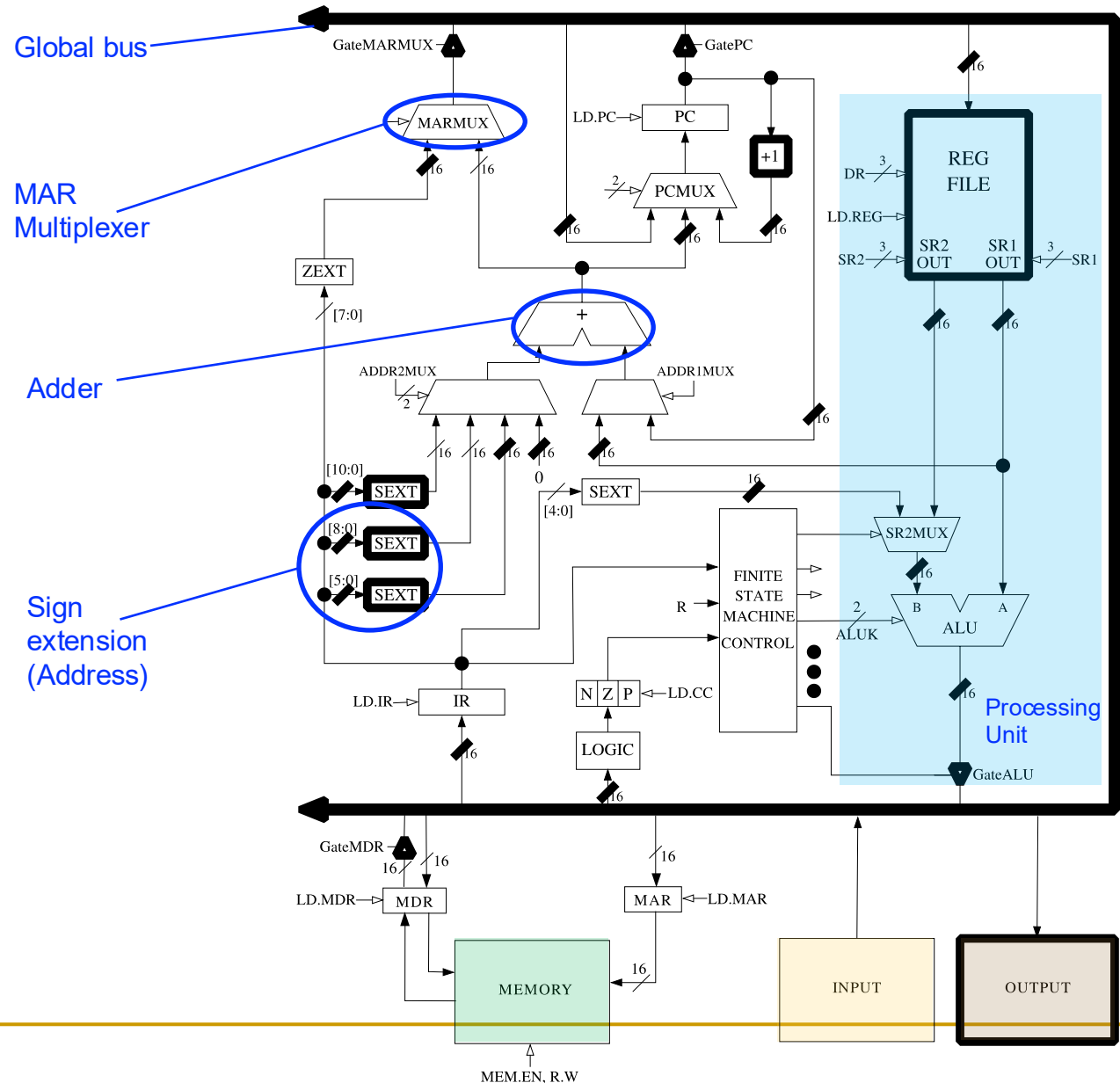
Machine Code

OP				DR		BaseR		offset6							
0 1 1 0				0 0 1		0 1 0		0 1 1 1 0 1							
15		12		11		9		8		6		5		0	



Again, the address of the operand can be **anywhere in the memory**

Address Calculation in LC-3 Data Path



Base+Offset Addressing Mode in MIPS

- In MIPS, **lw** and **sw** use base+offset mode (or **base addressing mode**)

High-level code

```
A[2] = a;
```

MIPS assembly

```
sw    $s3, 8($s0)
```

Memory[\$s0 + 8] ← \$s3

Field Values

op	rs	rt	imm
43	16	19	8

- imm** is the 16-bit offset, which is **sign-extended to 32 bits**

An Example Program in MIPS and LC-3

High-level code

```
a      = A[0];  
c      = a + b - 5;  
B[0]   = c;
```

MIPS registers

```
A = $s0  
b = $s2  
B = $s1
```

LC-3 registers

```
A = R0  
b = R2  
B = R1
```

MIPS assembly

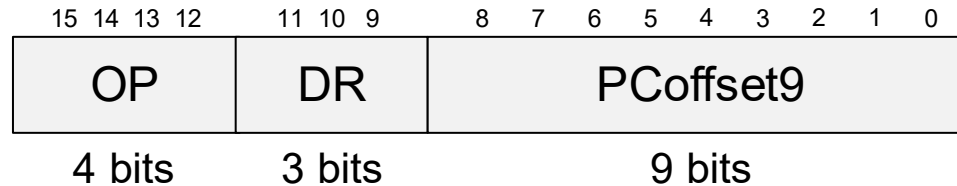
```
lw    $t0, 0($s0)  
add   $t1, $t0, $s2  
addi  $t2, $t1, -5  
sw    $t2, 0($s1)
```

LC-3 assembly

```
LDR   R5, R0, #0  
ADD   R6, R5, R2  
ADD   R7, R6, #-5  
STR   R7, R1, #0
```

Immediate Addressing Mode (in LC-3)

■ LEA (Load Effective Address)



- OP = 1110
- DR = destination register
- LEA: $DR \leftarrow PC^{\dagger} + \text{sign-extend}(\text{PCoffset9})$

What is the **difference from PC-Relative** addressing mode?

Answer: Instructions with **PC-Relative** mode **load from memory**, but **LEA does not** → Hence the name *Load Effective Address*

[†] This is the incremented PC

LEA in LC-3

LEA assembly and machine code

LC-3 assembly

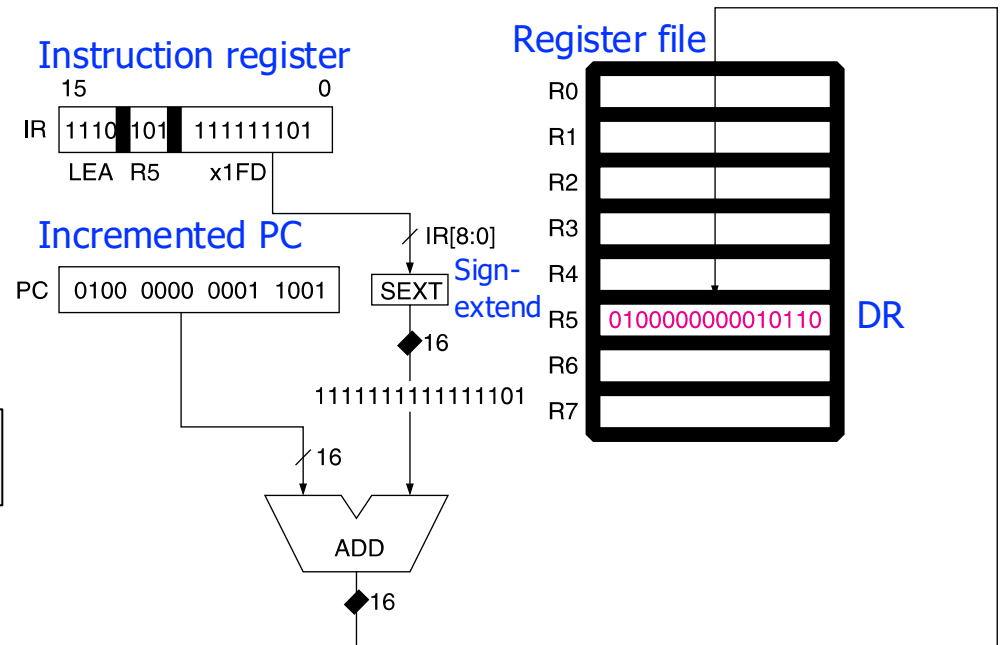
```
LEA R5, #-3
```

Field Values

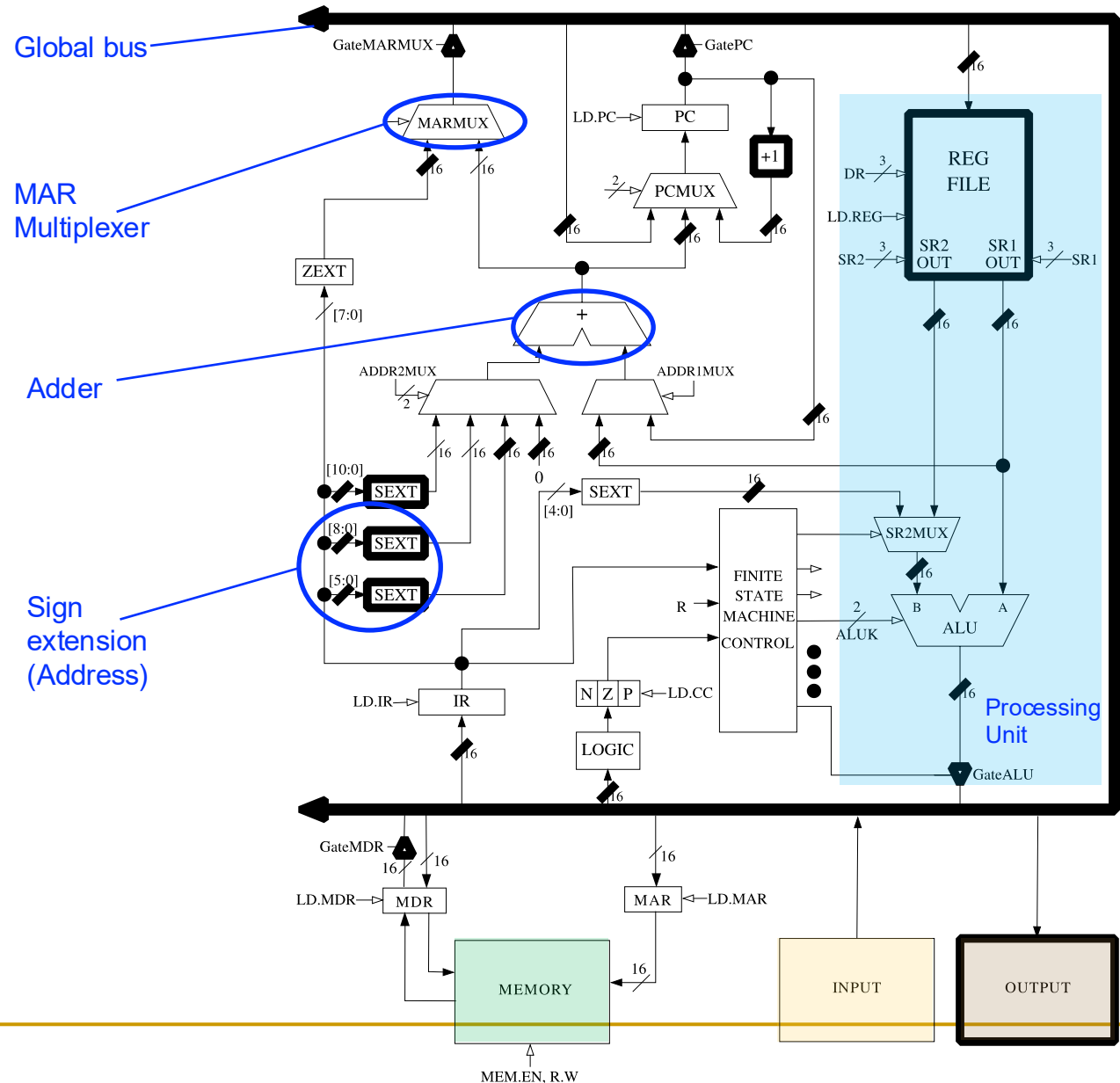
OP	DR	PCOffset9
E	5	0x1FD

Machine Code

OP	DR	PCOffset9
1 1 1 0	1 0 1	1 1 1 1 1 1 1 0 1
15	12	11 9 8 0



Address Calculation in LC-3 Data Path



Immediate Addressing Mode in MIPS

- In MIPS, **lui** (load upper immediate) loads a 16-bit immediate into the upper half of a register and sets the lower half to 0
- It is used to assign 32-bit constants to a register

High-level code

```
a = 0x6d5e4f3c;
```

MIPS assembly

```
# $s0 = a  
lui    $s0, 0x6d5e  
ori    $s0, 0x4f3c
```

Addressing Example in LC-3

- What is the final value of R3?

P&P, Chapter 5.3.5

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	1	0	1		R1 ← PC - 3
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	1	0	R2 ← R1 + 14
x30F8	0	0	1	1	0	1	0	1	1	1	1	1	1	0	1	1	M[x30F4] ← R2
x30F9	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 ← 0
x30FA	0	0	0	1	0	1	0	0	1	0	1	0	0	1	0	1	R2 ← R2 + 5
x30FB	0	1	1	1	0	1	0	0	0	1	0	0	1	1	1	0	M[R1 + 14] ← R2
x30FC	1	0	1	0	0	1	1	1	1	1	1	0	1	1	1	1	R3 ← M[M[x30F04]]

LC- ISA Instruction Encodings

Patt & Patel,
Back cover inside

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			0	00		SR2		
ADD ⁺	0001				DR			SR1			1	imm5				
AND ⁺	0101				DR			SR1			0	00		SR2		
AND ⁺	0101				DR			SR1			1	imm5				
BR	0000				n	z	p	PCOffset9								
JMP	1100				000			BaseR			000000					
JSR	0100				1	PCOffset11										
JSRR	0100				0	00		BaseR			000000					
LD ⁺	0010				DR			PCOffset9								
LDI ⁺	1010				DR			PCOffset9								
LDR ⁺	0110				DR			BaseR			offset6					
LEA ⁺	1110				DR			PCOffset9								
NOT ⁺	1001				DR			SR			111111					
RET	1100				000			111			000000					
RTI	1000				000000000000											
ST	0011				SR			PCOffset9								
STI	1011				SR			PCOffset9								
STR	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
reserved	1101															

Figure 5.3 Formats of the entire LC-3 instruction set. NOTE: ⁺ indicates instructions that modify condition codes

Addressing Example in LC-3

- What is the final value of R3?

P&P, Chapter 5.3.5

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	1	1	0	1	$R1 = PC - 3 = 0x30F7 - 3 = 0x30F4$
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	1	1	$R2 = R1 + 14 = 0x30F4 + 14 = 0x3102$
x30F8	0	1	1	1	0	1	0	1	1	1	1	1	1	0	1	1	$M[PC - 5] = M[0x30F4] = R2 = 0x3102$
x30F9	1	0	1	1	0	1	0	0	1	0	1	0	0	0	0	0	$R2 \leq 0$
x30FA	0	0	1	1	0	1	0	0	1	0	1	1	0	1	0	1	$R2 \leq R2 + 5 = 5$
x30FB	1	1	1	1	0	1	0	0	0	1	1	1	1	1	1	1	$M[R1 + 14] = M[0x30F4 + 14] = M[0x3102] = 5$
x30FC	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	$R3 = M[M[PC - 9]] = M[M[0x30FD - 9]] =$ $M[M[0x30F4]] = M[0x3102] = 5$

- The final value of R3 is 5

Control Flow Instructions

Control Flow Instructions

- Allow a program to execute **out of sequence**
- Conditional branches and unconditional jumps
 - **Conditional branches** are used to **make decisions**
 - E.g., if-else statement
 - In LC-3, three **condition codes** are used
 - **Jumps** are used to implement
 - **Loops**
 - **Function calls**
 - **JMP** in LC-3 and **j** in MIPS
 - We have already seen these

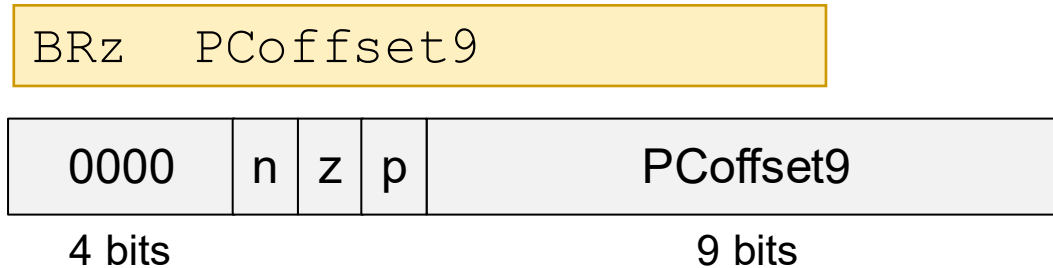
Conditional Control Flow (Conditional Branching)

Condition Codes in LC-3

- Each time one GPR (R0-R7) is written, **three single-bit registers** are updated
- Each of these **condition codes** are either set (set to 1) or cleared (set to 0)
 - If the written value is **negative**
 - **N** is set, Z and P are cleared
 - If the written value is **zero**
 - **Z** is set, N and P are cleared
 - If the written value is **positive**
 - **P** is set, N and Z are cleared
- x86 and SPARC are examples of ISAs that use condition codes

Conditional Branches in LC-3

■ BRz (Branch if Zero)



- $n, z, p =$ **which condition code is tested** (N, Z, and/or P)
 - n, z, p : instruction bits to identify the condition codes to be tested
 - N, Z, P : values of the corresponding condition codes
- $PCoffset9 =$ immediate or constant value
- if $((n \text{ AND } N) \text{ OR } (p \text{ AND } P) \text{ OR } (z \text{ AND } Z))$
 - then $PC \leftarrow PC^{\dagger} + \text{sign-extend}(PCoffset9)$
- Variations: BRn, BRz, BRp, BRzp, BRnp, BRnz, BRnzp

[†] This is the incremented PC

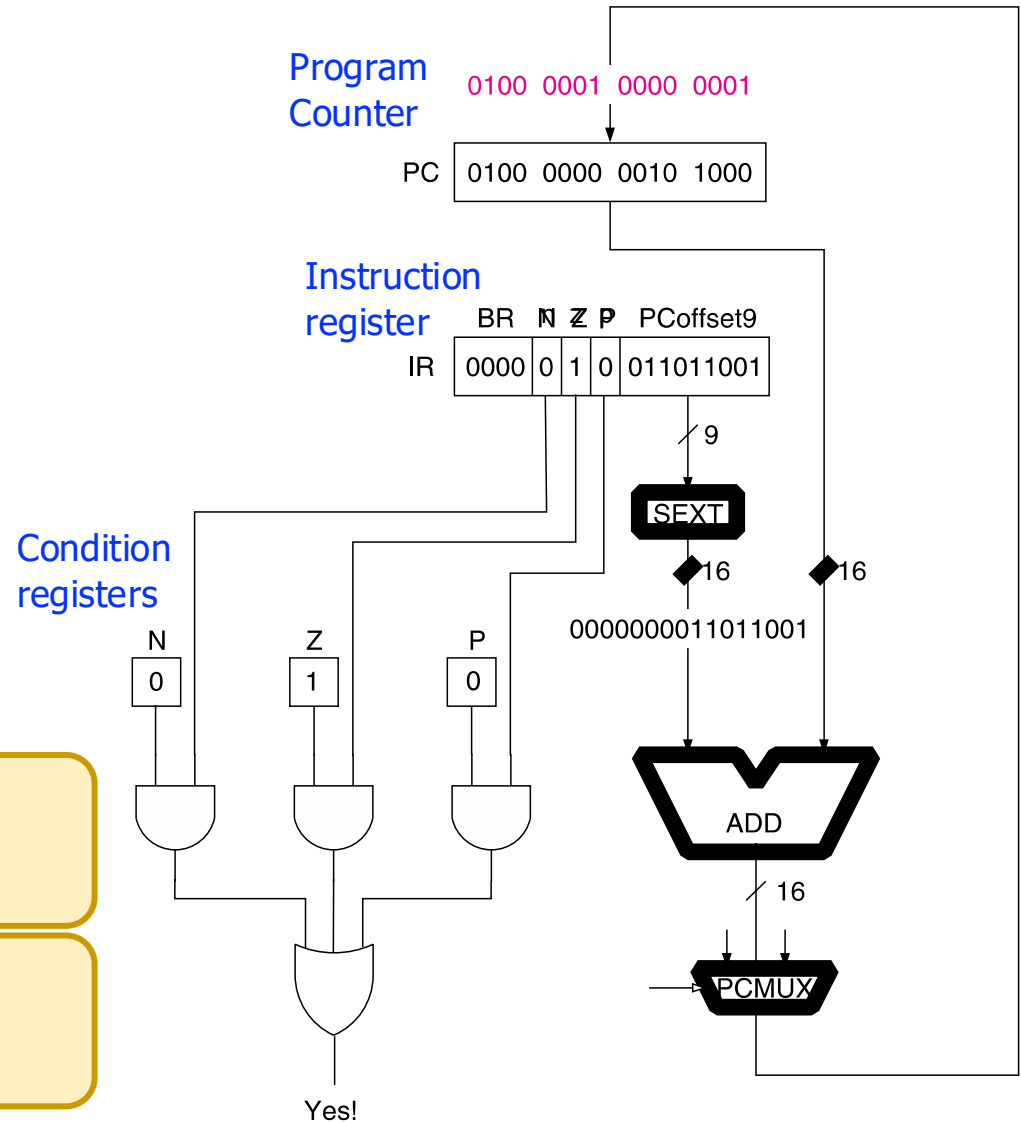
Conditional Branches in LC-3

■ BRz

BRz 0x0D9

What if $n = z = p = 1$?*
(i.e., BRnzp)

And what if $n = z = p = 0$?

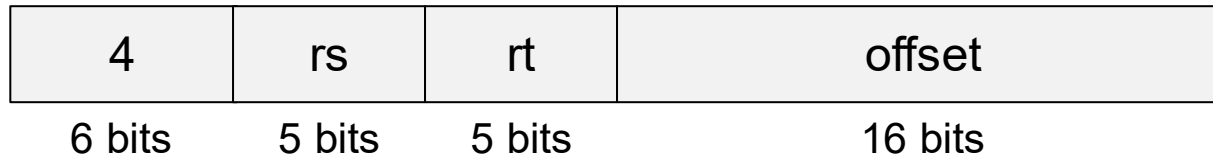


*n, z, p are the instruction bits to identify the condition codes to be tested

Conditional Branches in MIPS

■ beq (Branch if Equal)

```
beq  $s0, $s1, offset
```



- 4 = opcode
- rs, rt = source registers
- offset = immediate or constant value
- if $rs == rt$
 - then $PC \leftarrow PC^{\dagger} + \text{sign-extend}(\text{offset}) * 4$
- Variations: beq, bne, blez, bgtz

[†] This is the incremented PC

Branch If Equal in MIPS and LC-3

MIPS assembly

```
beq  $s0, $s1, offset
```

LC-3 assembly

```
NOT  R2, R1
```

```
ADD  R3, R2, #1
```

```
ADD  R4, R3, R0
```

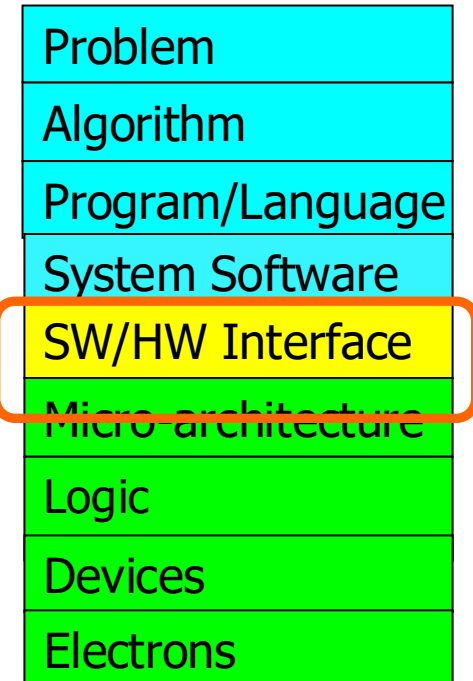
```
BRz  offset
```

**Subtract
(R0 - R1)**

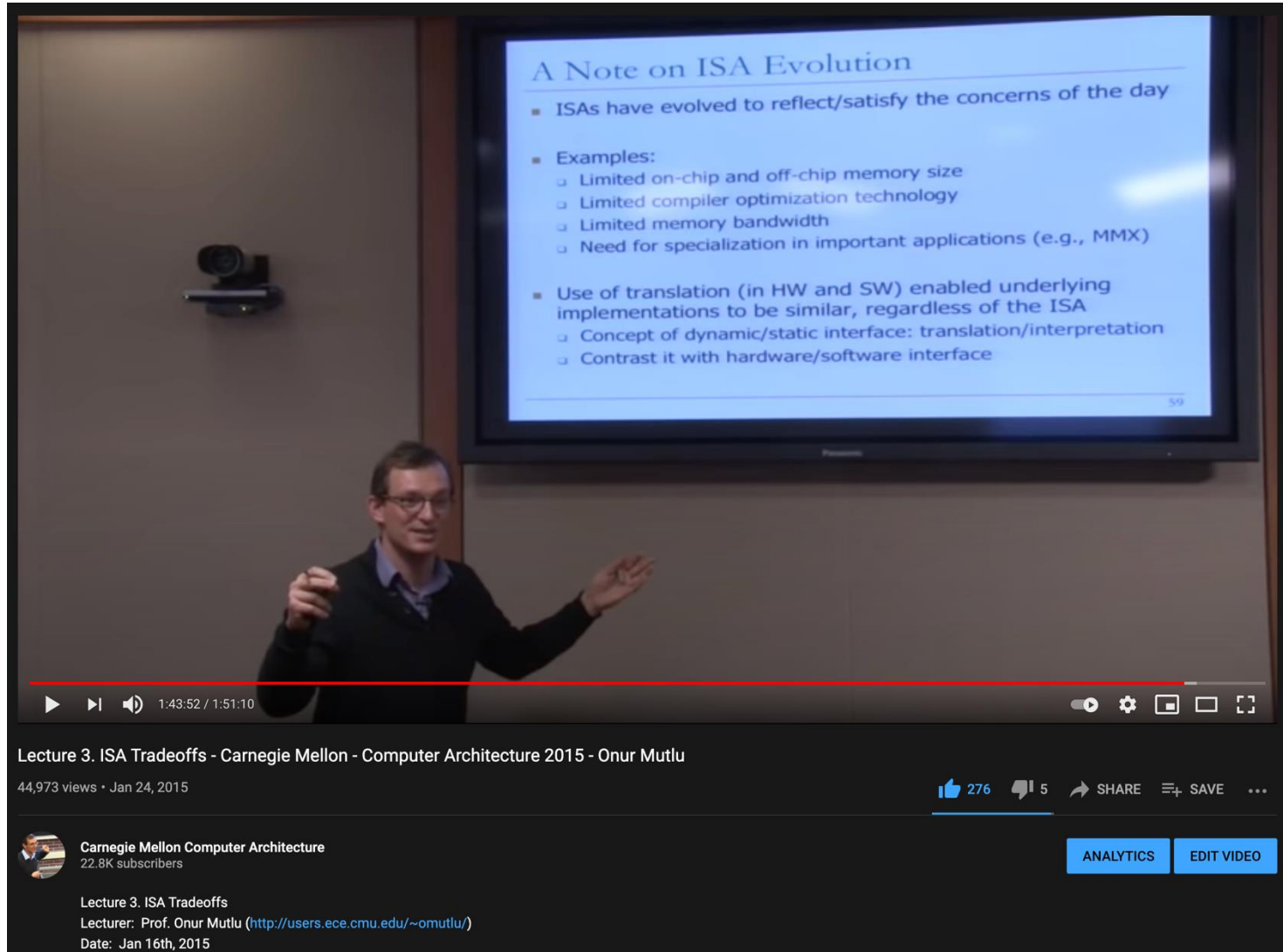
- This is an example of **tradeoff** in the instruction set
 - ❑ The same functionality requires **more instructions in LC-3**
 - ❑ But, the **control logic** requires **more complexity in MIPS**

What We Learned

- Basic elements of a computer & the von Neumann model
 - LC-3: An example von Neumann machine
- Instruction Set Architectures: LC-3 and MIPS
 - Operate instructions
 - Data movement instructions
 - Control instructions
- Instruction formats
- Addressing modes



There Is A Lot More to Cover on ISAs



A Note on ISA Evolution

- ISAs have evolved to reflect/satisfy the concerns of the day
- Examples:
 - Limited on-chip and off-chip memory size
 - Limited compiler optimization technology
 - Limited memory bandwidth
 - Need for specialization in important applications (e.g., MMX)
- Use of translation (in HW and SW) enabled underlying implementations to be similar, regardless of the ISA
 - Concept of dynamic/static interface: translation/interpretation
 - Contrast it with hardware/software interface

59

Lecture 3. ISA Tradeoffs - Carnegie Mellon - Computer Architecture 2015 - Onur Mutlu

44,973 views • Jan 24, 2015

Carnegie Mellon Computer Architecture
22.8K subscribers

Lecture 3. ISA Tradeoffs
Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)
Date: Jan 16th, 2015

<https://www.youtube.com/onurmutlulectures>

Many Different ISAs Over Decades

- x86
- PDP-x: Programmed Data Processor (PDP-11)
- VAX
- IBM 360
- CDC 6600
- SIMD ISAs: CRAY-1, Connection Machine
- VLIW ISAs: Multiflow, Cydrome, IA-64 (EPIC)
- PowerPC, POWER
- RISC ISAs: Alpha, MIPS, SPARC, ARM, RISC-V, ...
- What are the fundamental differences?
 - E.g., how instructions are specified and what they do
 - E.g., how complex are instructions, data types, addr. modes

Complex vs. Simple Instructions+Data Types

- **Complex instruction**: An instruction **does a lot of work**, e.g. many operations
 - ❑ Insert in a doubly linked list
 - ❑ Compute FFT
 - ❑ String copy
 - ❑ Matrix multiply
 - ❑ ...
- **Simple instruction**: An instruction **does little work** -- it is a primitive using which complex operations can be built
 - ❑ Add
 - ❑ XOR
 - ❑ Multiply
 - ❑ ...

Complex vs. Simple Instructions+Data Types

■ Advantages of Complex Instructions + Data Types

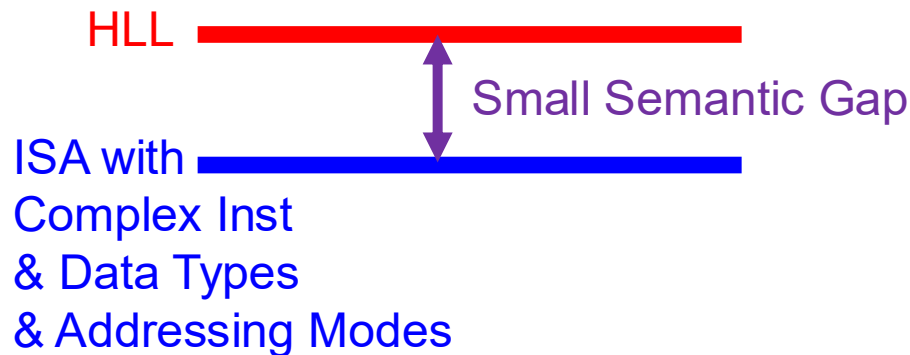
- + **Denser encoding** → smaller code size → better memory utilization, saves off-chip bandwidth, better cache hit rate (better packing of instructions)
- + **Simpler compiler**: no need to optimize small instructions as much

■ Disadvantages of Complex Instructions + Data Types

- **Larger chunks of work** → compiler has less opportunity to optimize (limited in fine-grained optimizations it can do)
- **More complex hardware** → translation from a high level to control signals and optimization needs to be done by hardware

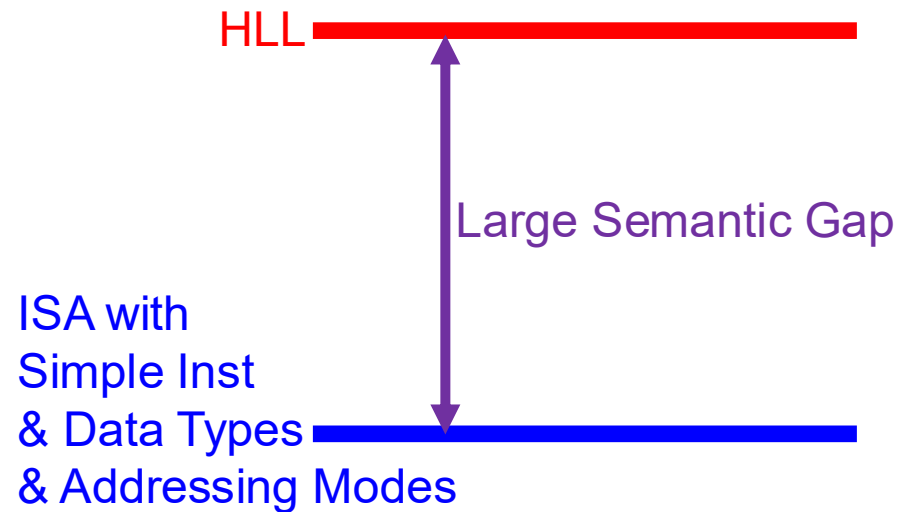
Semantic Gap

- How close instructions & data types & addressing modes are to high-level language (HLL)



HW
Control
Signals

Easier mapping of HLL to ISA
Less work for software designer
More work for hardware designer
Optimization burden on HW



HW
Control
Signals

Harder mapping of HLL to ISA
More work for software designer
Less work for hardware designer
Optimization burden on SW

We Covered Until Here in Lecture

Digital Design & Computer Arch.

Lecture 8: Instruction Set Architectures II

Prof. Onur Mutlu

ETH Zürich

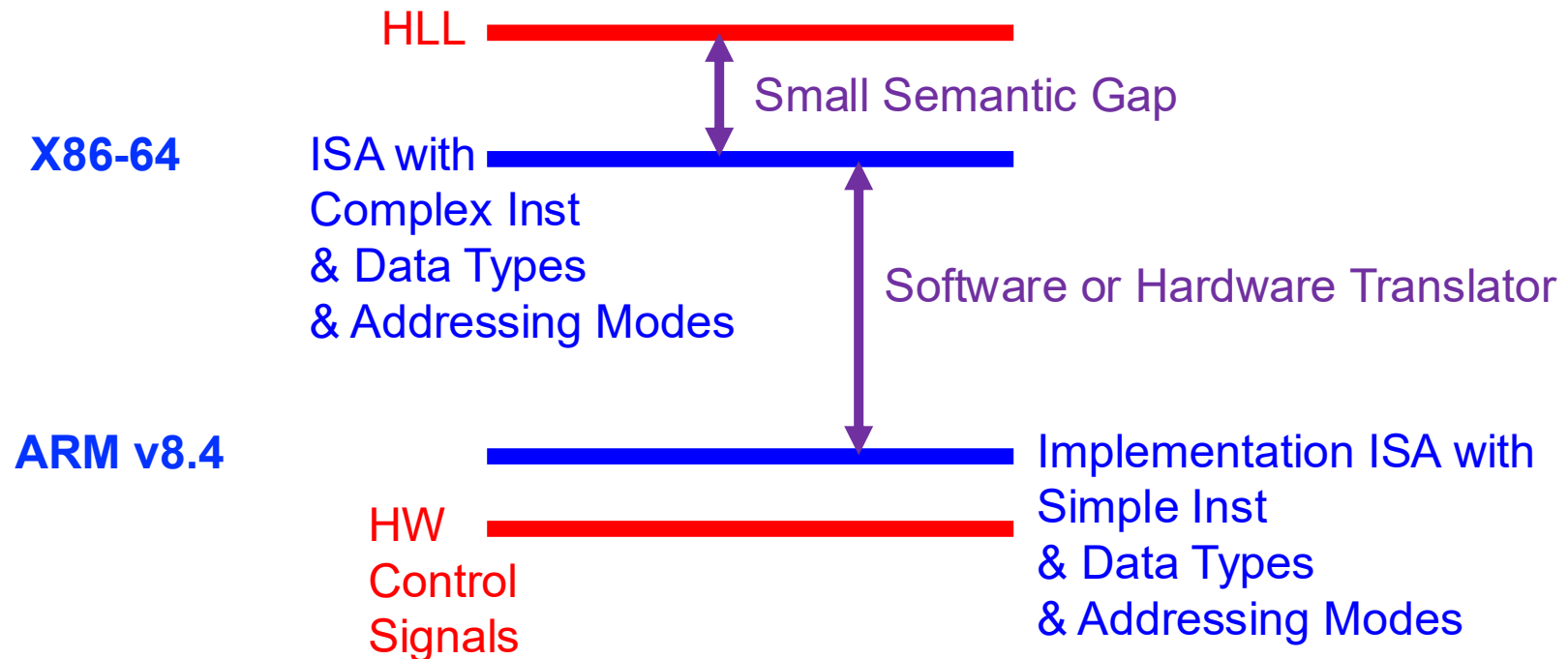
Spring 2026

13 March 2026

**Further Slides for Your Own Study
(May Be Covered in Future Lectures)**

How to Change the Semantic Gap Tradeoffs

- Translate from one ISA into a different “implementation” ISA



An Example: Rosetta 2 Binary Translator

Rosetta 2 [\[edit \]](#)

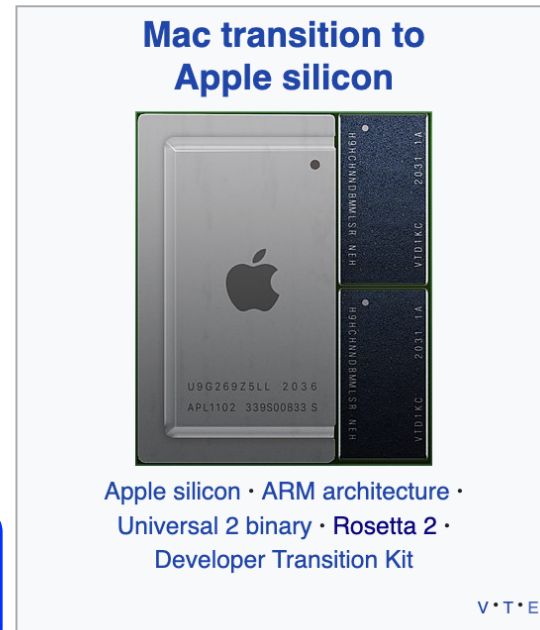
In 2020, Apple announced Rosetta 2 would be bundled with [macOS Big Sur](#), to aid in the [Mac transition to Apple silicon](#). The software permits many applications compiled exclusively for execution on x86-64-based processors to be translated for execution on Apple silicon.^{[2][8]}

In addition to the [just-in-time](#) (JIT) translation support, Rosetta 2 offers [ahead-of-time compilation](#) (AOT), with the x86-64 code fully translated, just once, when an application without a universal binary is installed on an Apple silicon Mac.^[9]

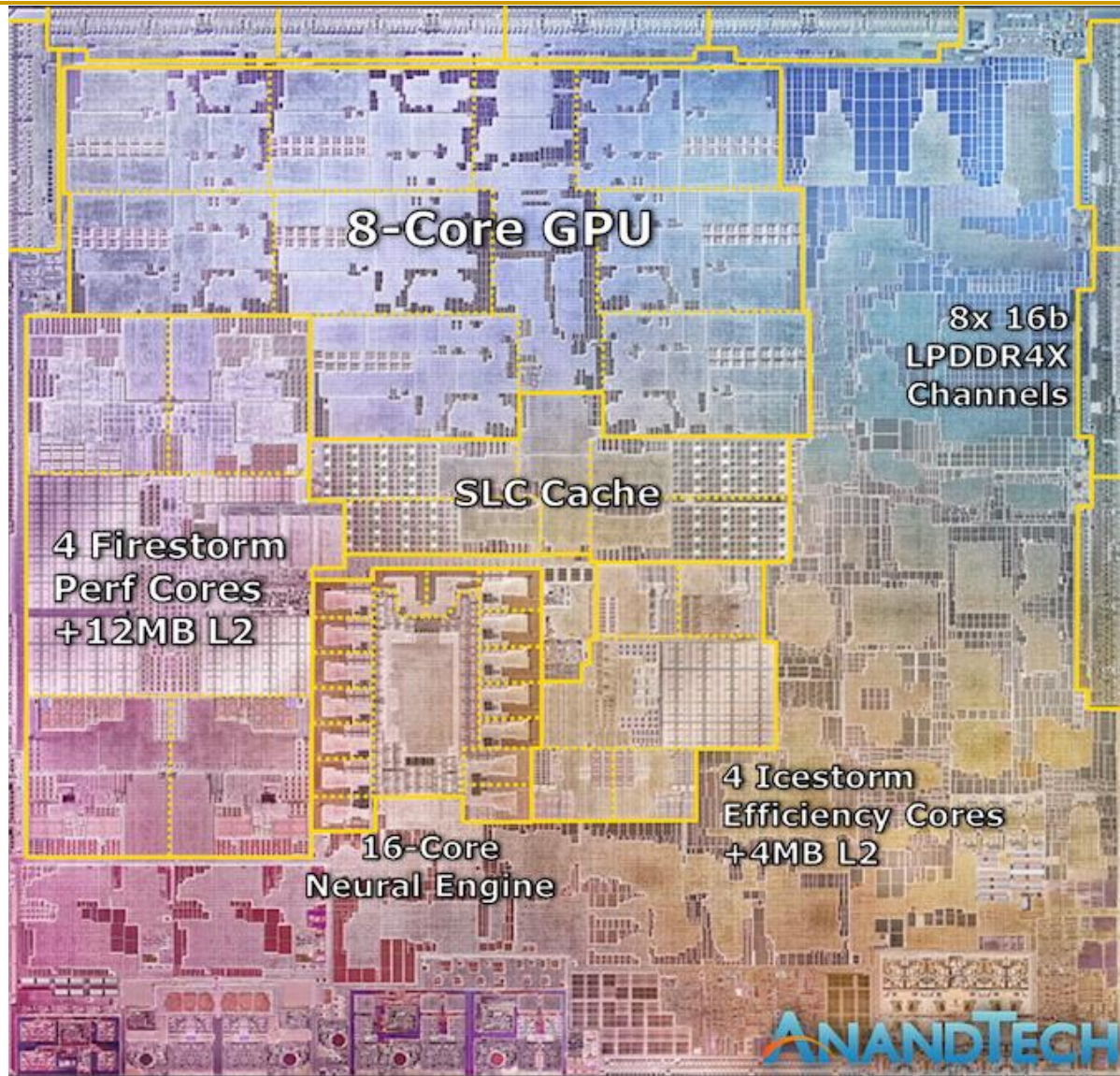
Rosetta 2's performance has been praised greatly.^{[10][11]} In some benchmarks, x86-64-only programs performed better under Rosetta 2 on a Mac with an Apple M1 SOC than natively on a Mac with an Intel x86-64 processor. One of the key reasons why Rosetta 2 provides such high level of translation efficiency is the support of x86-64 [memory ordering](#) in Apple M1 SOC.^[12]

Although Rosetta 2 works for most software, some software doesn't work at all^[13] or is reported to be "sluggish".^[14] A lot of software can be made compatible with the new Macs by the vendor recompiling the software, often a simple task; while for some software (such as software that includes [assembly language](#) code, or that generates [machine code](#)), the changes to make them work aren't simple and cannot be automated.

Similar to the first version, Rosetta 2 does not normally require user intervention. When a user attempts to launch an x86-64-only application for the first time, macOS prompts them to install Rosetta 2 if it is not already available. Subsequent launches of x86-64 programs will execute via translation automatically. An option also exists to force a universal binary to run as x86-64 code through Rosetta 2, even on an ARM-based machine.^[15]

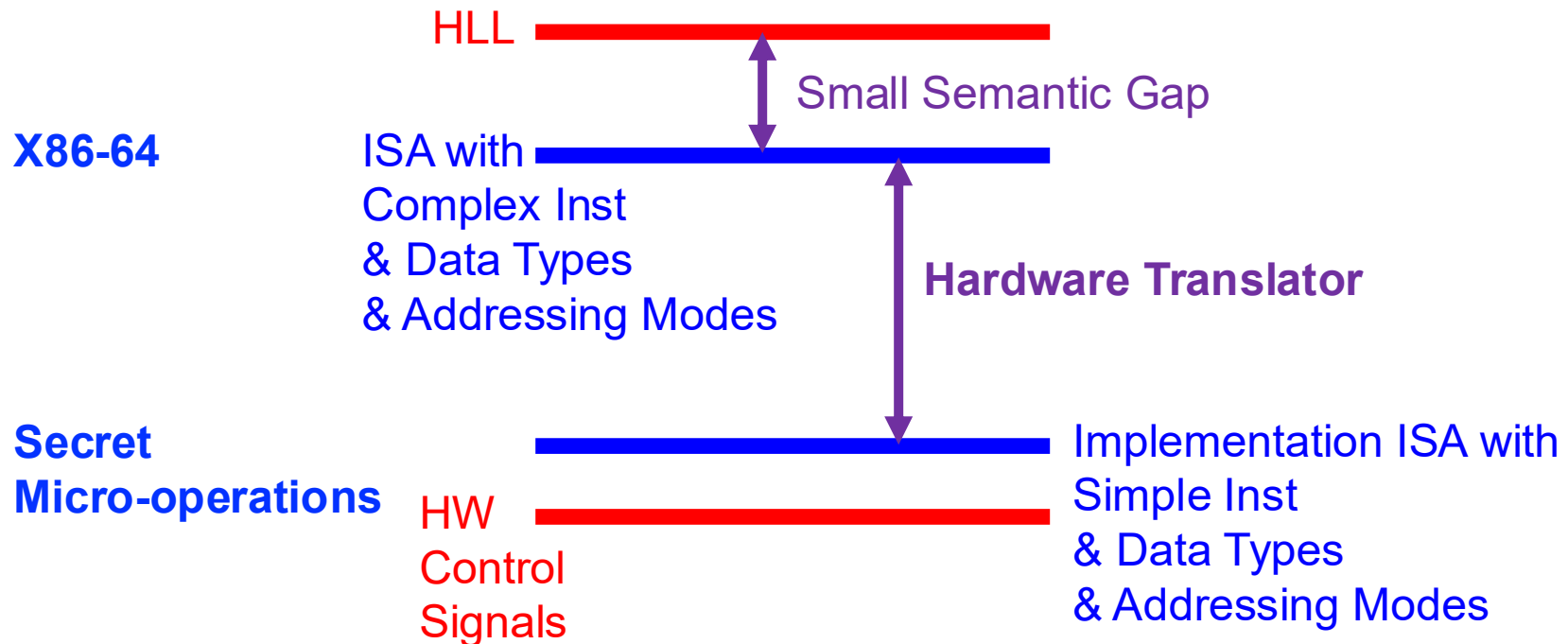


An Example: Rosetta 2 Binary Translator



Apple M1,
2021

Another Example: Intel and AMD Processors



Another Example: Intel and AMD Processors

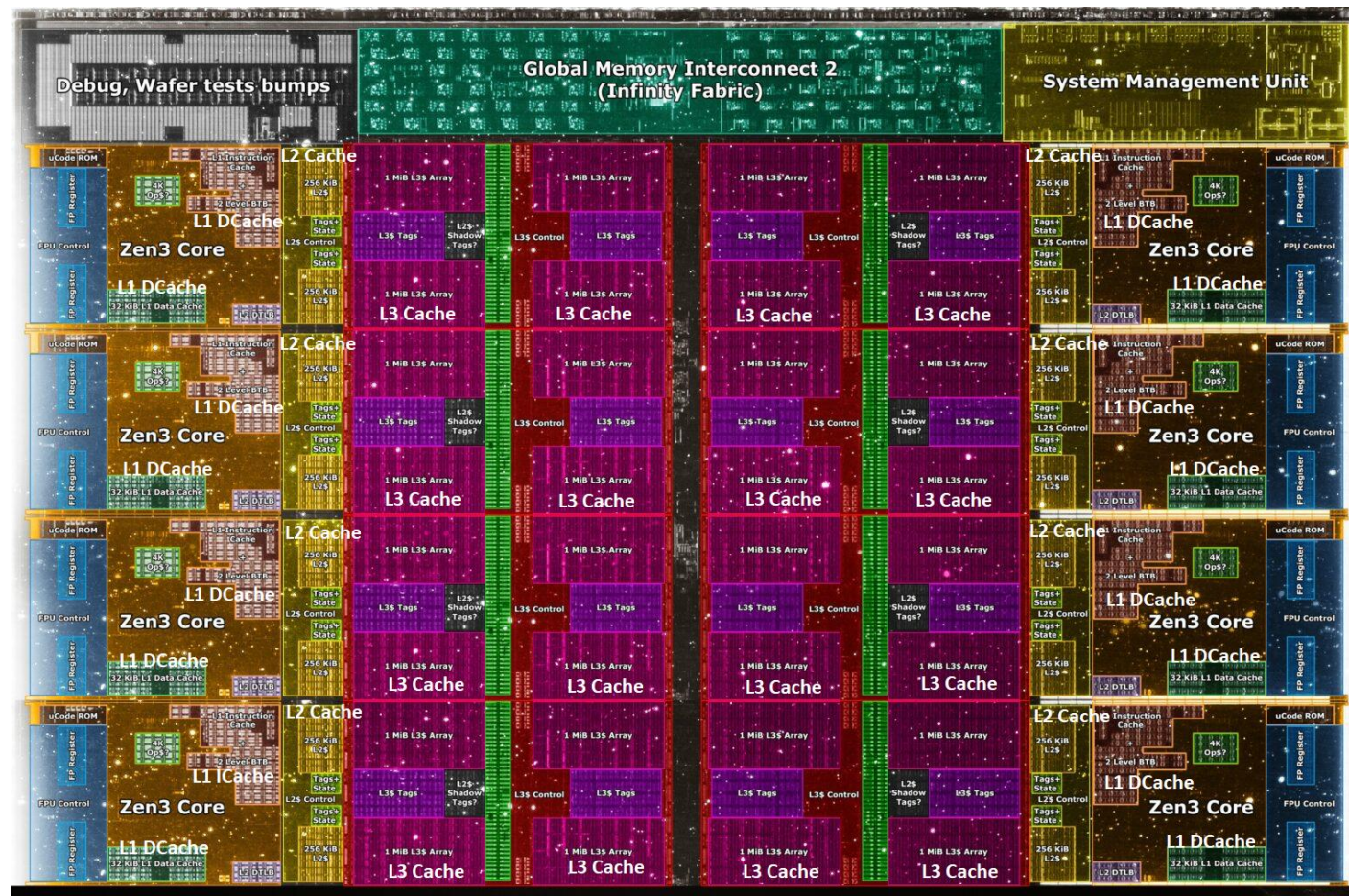


10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>

Die shot interpretation by Locuza, October 2021

Intel Alder Lake,
2021

Another Example: Intel and AMD Processors



Core Count:
8 cores/16 threads

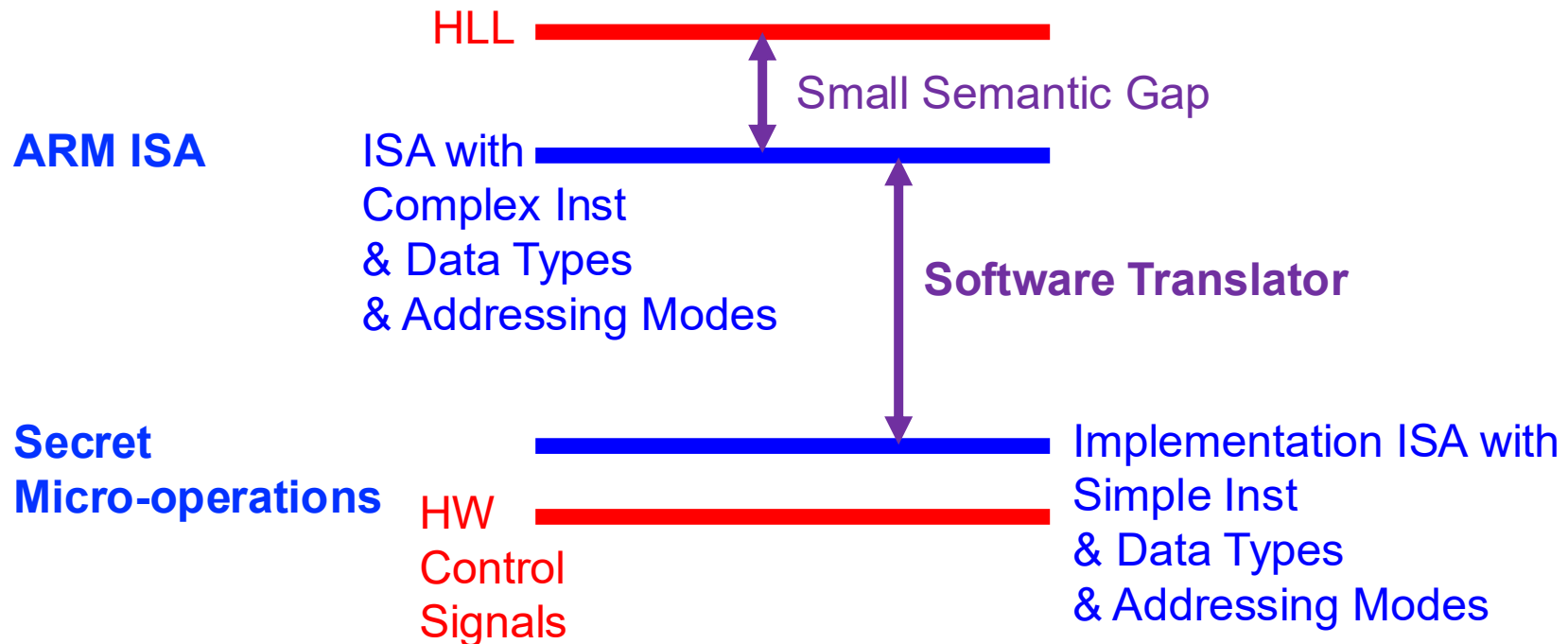
L1 Caches:
32 KB per core

L2 Caches:
512 KB per core

L3 Cache:
32 MB shared

AMD Ryzen 5000, 2020

Another Example: NVIDIA Denver



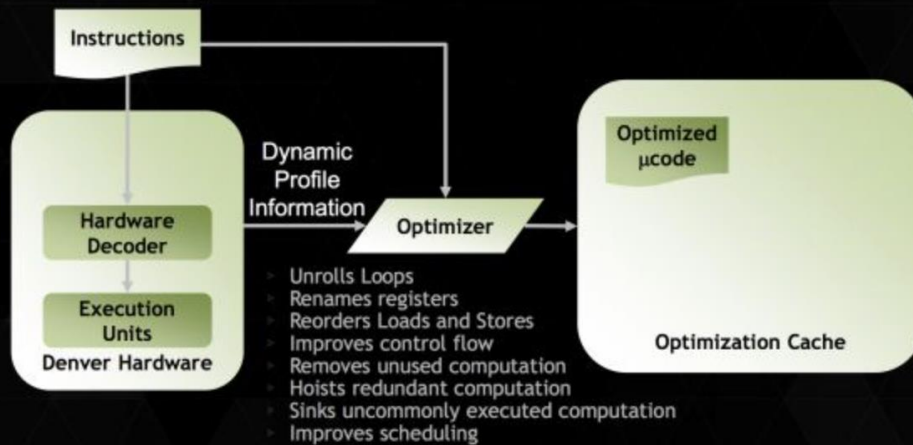
Another Example: NVIDIA Denver

The Secret of Denver: Binary Translation & Code Optimization

As we alluded to earlier, NVIDIA's decision to forgo a traditional out-of-order design for Denver means that much of Denver's potential is contained in its software rather than its hardware. The underlying chip itself, though by no means simple, is at its core a very large in-order processor. So it falls to the software stack to make Denver sing.

Accomplishing this task is NVIDIA's dynamic code optimizer (DCO). The purpose of the DCO is to accomplish two tasks: to translate ARM code to Denver's native format, and to optimize this code to make it run better on Denver. With no out-of-order hardware on Denver, it is the DCO's task to find instruction level parallelism within a thread to fill Denver's many execution units, and to reorder instructions around potential stalls, something that is no simple task.

DYNAMIC CODE OPTIMIZATION OPTIMIZE ONCE, USE MANY TIMES



Transmeta: x86 to VLIW Translation

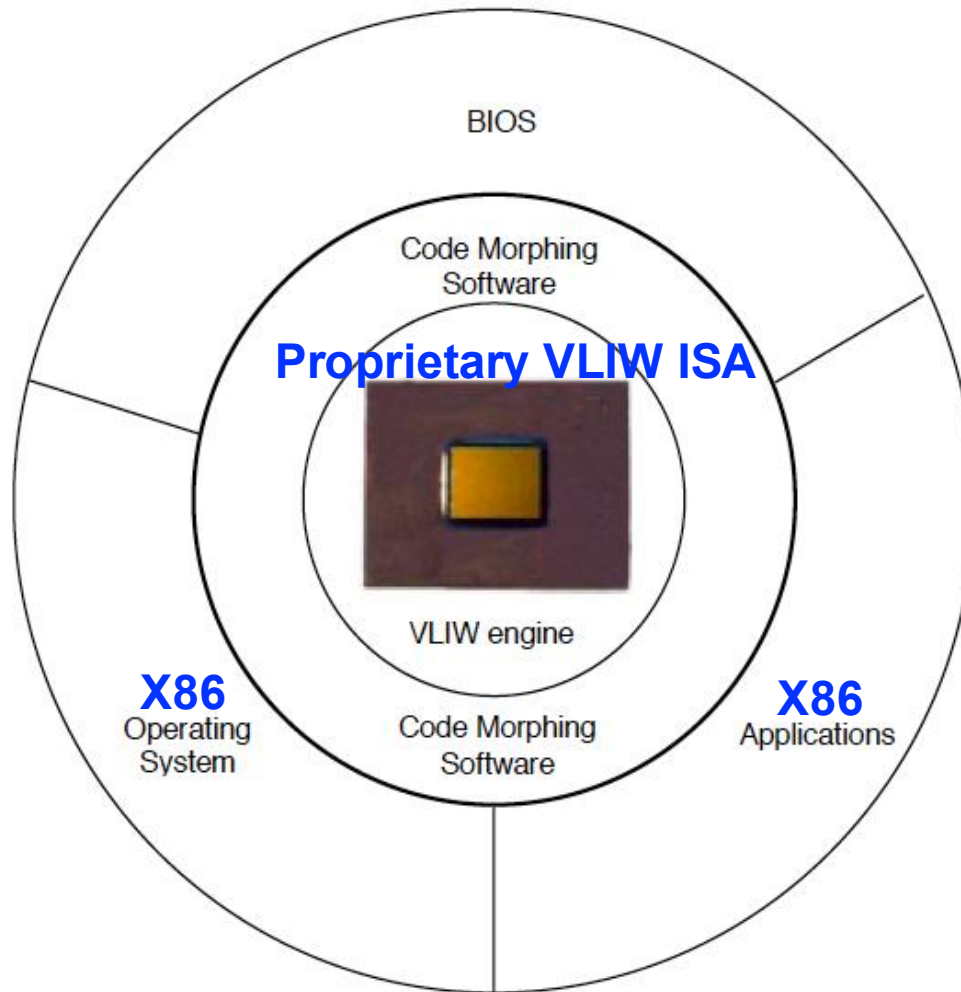


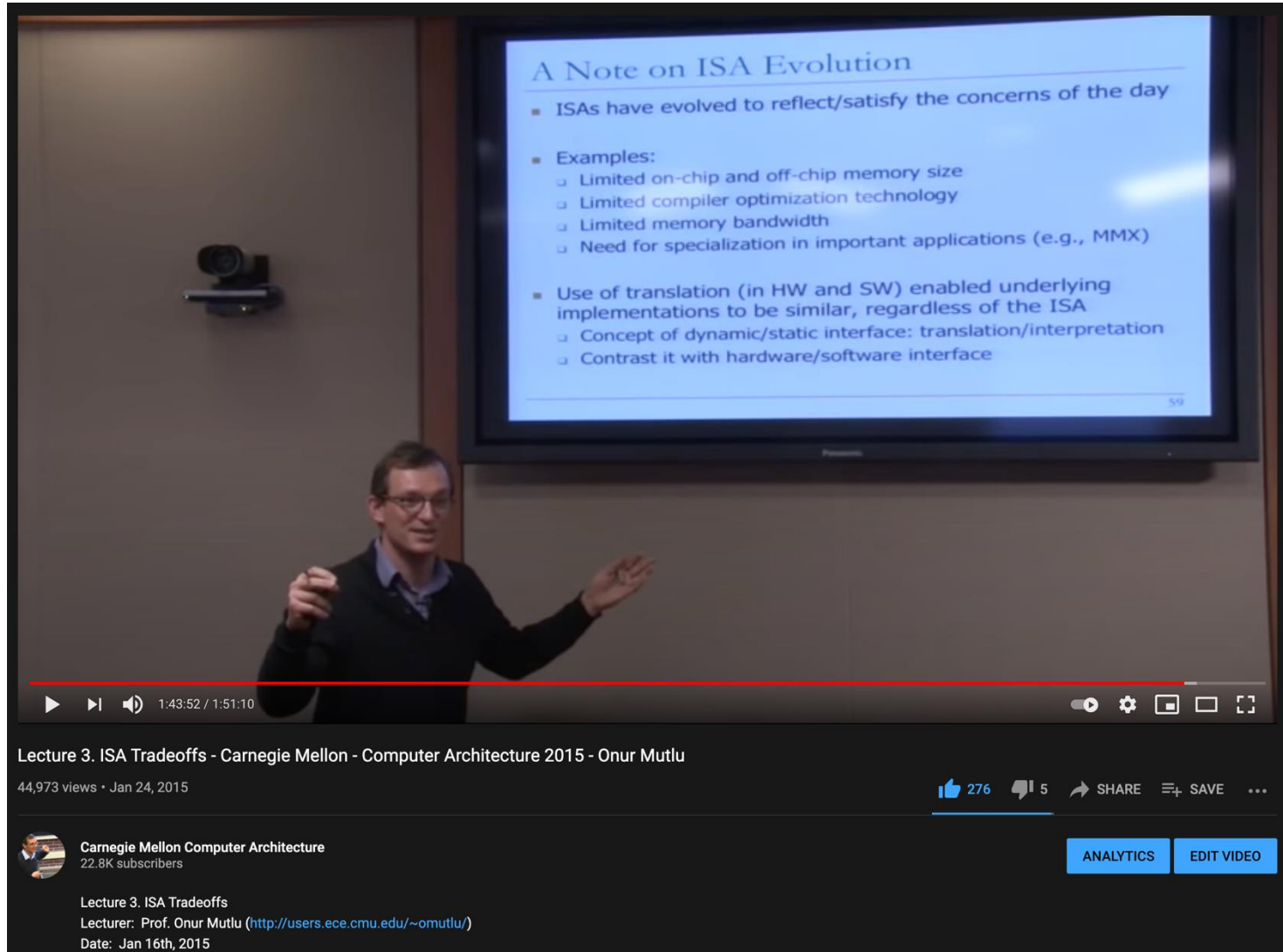
Figure 5. The Code Morphing software mediates between x86 software and the Crusoe processor.

Klaiber, “[The Technology Behind Crusoe Processors](#),” Transmeta White Paper 2000.

ISA-level Tradeoffs: Number of Registers

- Affects:
 - Number of bits used for encoding register address
 - Number of values kept in fast storage (register file)
 - (uarch) Size, access time, power consumption of register file
- Large number of registers:
 - + Enables better register allocation (and optimizations) by compiler → fewer saves/restores
 - Larger instruction size
 - Larger register file size

There Is A Lot More to Cover on ISAs



A Note on ISA Evolution

- ISAs have evolved to reflect/satisfy the concerns of the day
- Examples:
 - Limited on-chip and off-chip memory size
 - Limited compiler optimization technology
 - Limited memory bandwidth
 - Need for specialization in important applications (e.g., MMX)
- Use of translation (in HW and SW) enabled underlying implementations to be similar, regardless of the ISA
 - Concept of dynamic/static interface: translation/interpretation
 - Contrast it with hardware/software interface

59

1:43:52 / 1:51:10

Lecture 3. ISA Tradeoffs - Carnegie Mellon - Computer Architecture 2015 - Onur Mutlu

44,973 views • Jan 24, 2015

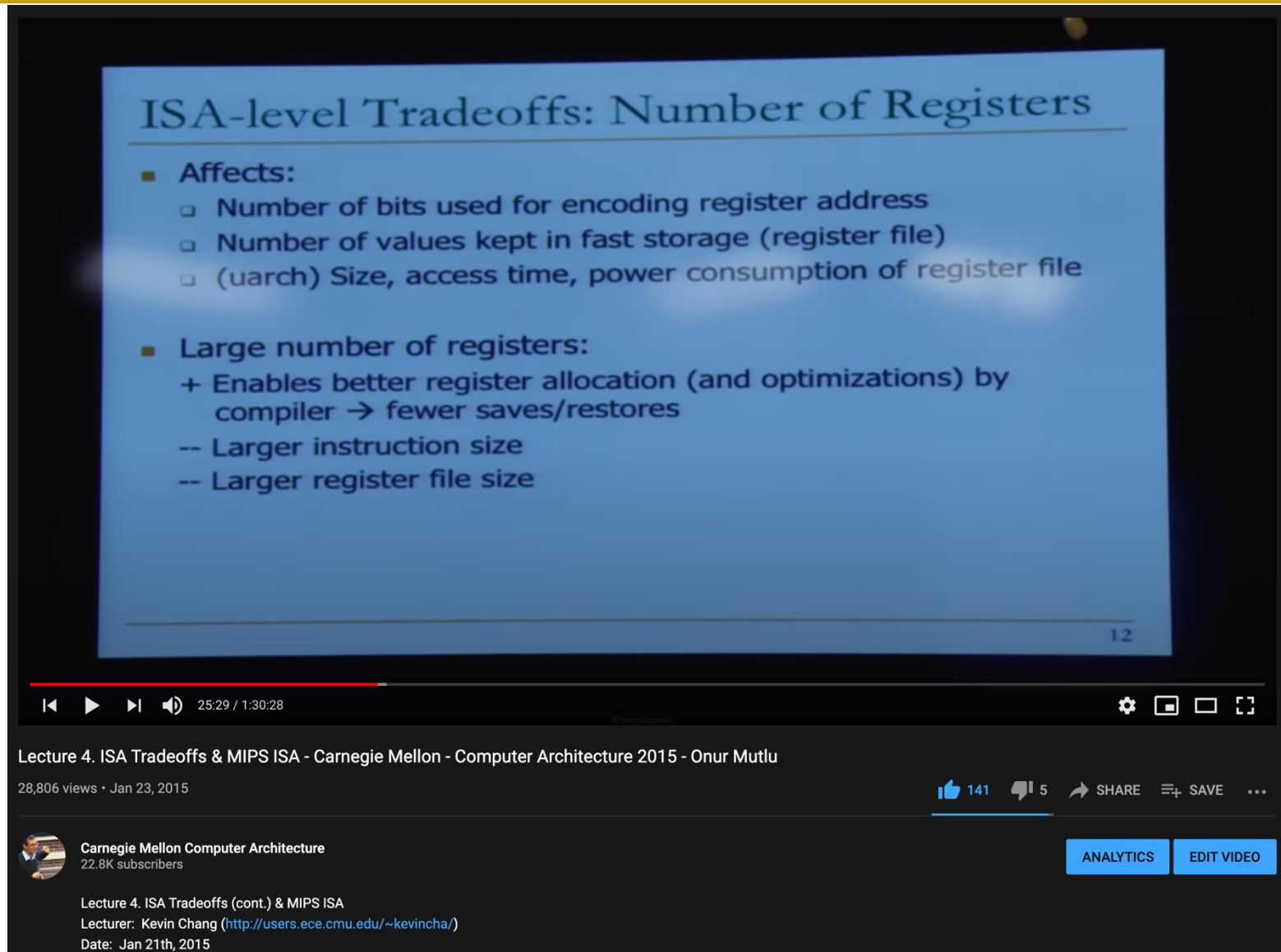
Carnegie Mellon Computer Architecture
22.8K subscribers

Lecture 3. ISA Tradeoffs
Lecturer: Prof. Onur Mutlu (<http://users.ece.cmu.edu/~omutlu/>)
Date: Jan 16th, 2015

276 5 SHARE SAVE ...

ANALYTICS EDIT VIDEO

There Is A Lot More to Cover on ISAs



The video player displays a slide with the following content:

ISA-level Tradeoffs: Number of Registers

- **Affects:**
 - Number of bits used for encoding register address
 - Number of values kept in fast storage (register file)
 - (uarch) Size, access time, power consumption of register file
- **Large number of registers:**
 - + Enables better register allocation (and optimizations) by compiler → fewer saves/restores
 - Larger instruction size
 - Larger register file size

12

Lecture 4. ISA Tradeoffs & MIPS ISA - Carnegie Mellon - Computer Architecture 2015 - Onur Mutlu

28,806 views • Jan 23, 2015

141 5 SHARE SAVE ...

 **Carnegie Mellon Computer Architecture**
22.8K subscribers

Lecture 4. ISA Tradeoffs (cont.) & MIPS ISA
Lecturer: Kevin Chang (<http://users.ece.cmu.edu/~kevincha/>)
Date: Jan 21th, 2015

ANALYTICS EDIT VIDEO

Detailed Lectures on ISAs & ISA Tradeoffs

■ Computer Architecture, Spring 2015, Lecture 3

- ISA Tradeoffs (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=QKdiZSfwg-g&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=3>

■ Computer Architecture, Spring 2015, Lecture 4

- ISA Tradeoffs & MIPS ISA (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=RBgeCCW5Hjs&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=4>

■ Computer Architecture, Spring 2015, Lecture 2

- Fundamental Concepts and ISA (CMU, Spring 2015)
- <https://www.youtube.com/watch?v=NpC39uS4K4o&list=PL5PHm2jkkXmi5CxxI7b3JCL1TWybTDtKq&index=2>

ISA Design and Tradeoffs: More Critical Thinking

The Von Neumann Model/Architecture

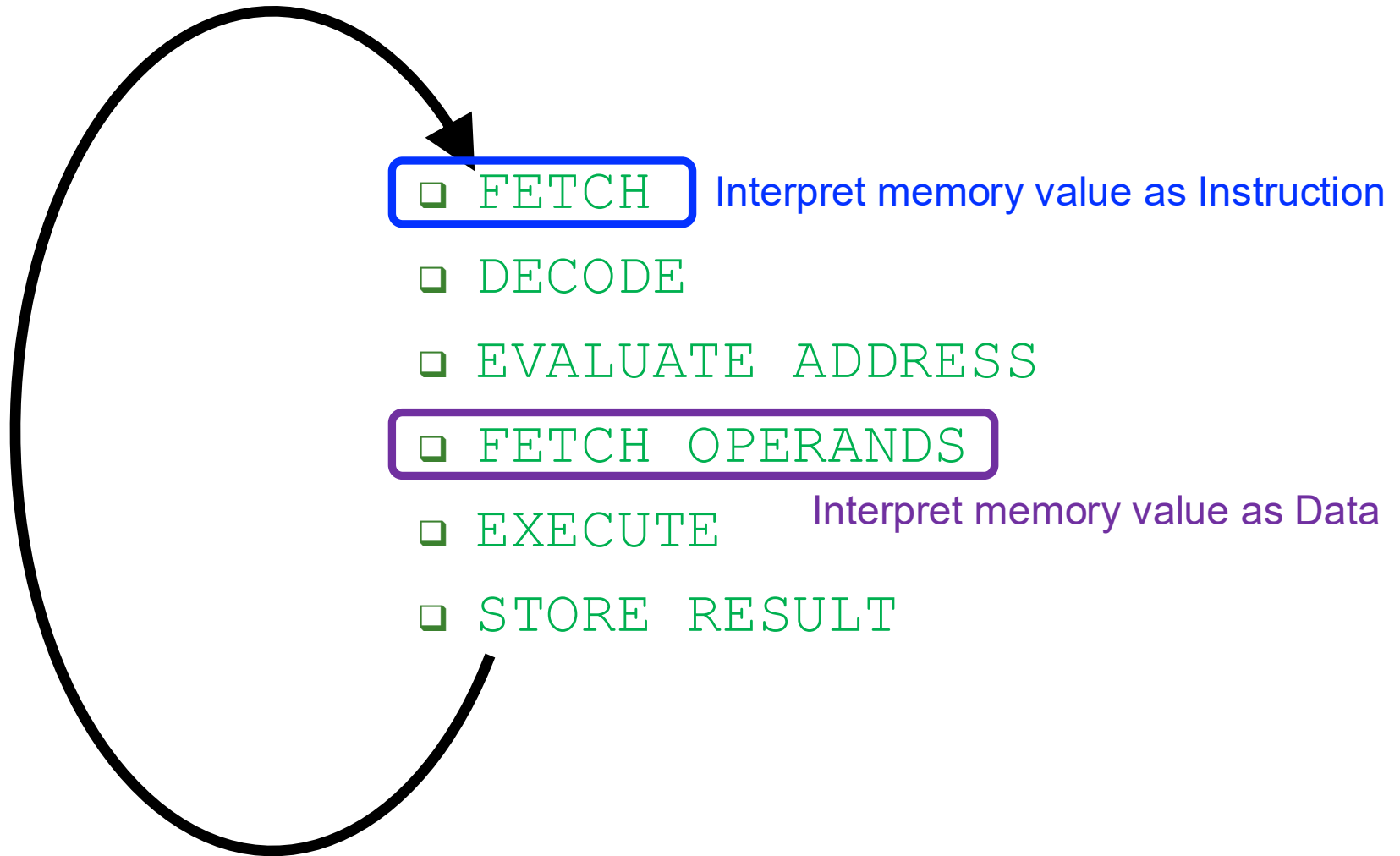
Stored program

Sequential instruction processing

The von Neumann Model/Architecture

- Von Neumann model is also called *stored program computer* (instructions in memory). It has two key properties:
- **Stored program**
 - Instructions stored in a linear memory array
 - **Memory is unified** between instructions and data
 - The interpretation of a stored value depends on the control signals
When is a value interpreted as an instruction?
- **Sequential instruction processing**

Recall: The Instruction Cycle



Whether a value fetched from memory is interpreted as an instruction depends on **when** that value is **fetched** in the instruction processing cycle.

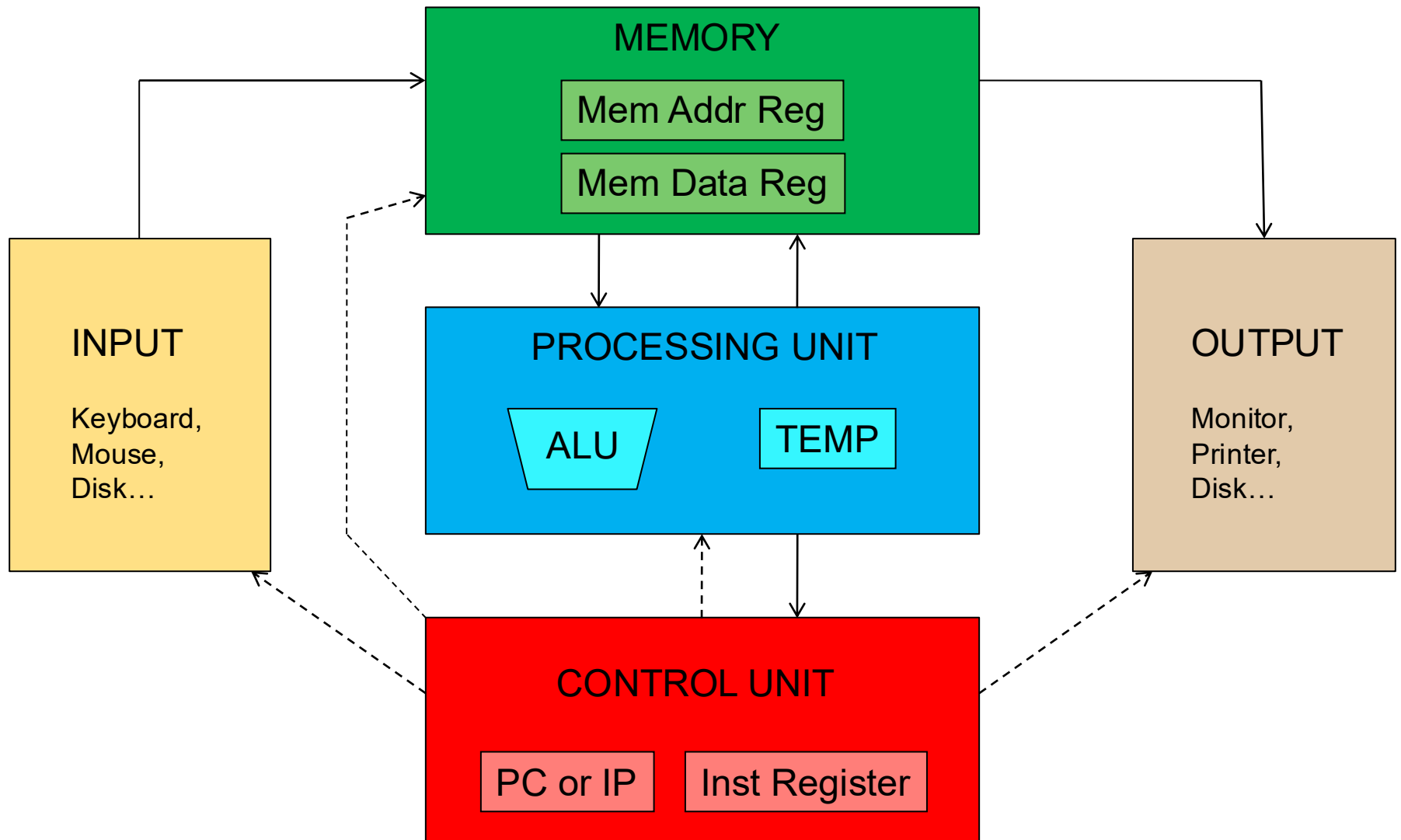
The von Neumann Model/Architecture

- Von Neumann model is also called *stored program computer* (instructions in memory). It has two key properties:
- **Stored program**
 - Instructions stored in a linear memory array
 - **Memory is unified** between instructions and data
 - **The interpretation of a stored value depends on the control signals**
When is a value interpreted as an instruction?
- **Sequential instruction processing**
 - One instruction processed (fetched, executed, completed) at a time
 - **Program counter (instruction pointer)** identifies the current instruction
 - **Program counter is advanced sequentially** except for control transfer instructions

The von Neumann Model/Architecture

- Recommended reading
 - Burks, Goldstein, von Neumann, “Preliminary discussion of the logical design of an electronic computing instrument,” 1946.
- Important reading
 - Patt and Patel book, Chapter 4, “The von Neumann Model”
- **Stored program**
- **Sequential instruction processing**

The Von Neumann Model (of a Computer)



The Von Neumann Model (of a Computer)

- Q: Is this the only way that a computer can process computer programs?

The von Neumann Model

- In order to build a computer, we need an execution model for processing computer programs

- John von Neumann proposed a fundamental model in 1946

- The von Neumann Model consists of 5 components

- Memory (stores the program and data)
- Processing unit
- Input
- Output
- Control unit (controls the order in which instructions are carried out)



- Throughout this lecture, we will examine two examples of the von Neumann model

- LC-3
- MIPS

Burks, Goldstein, von Neumann,
"Preliminary discussion of the logical design
of an electronic computing instrument," 1946.

All general-purpose computers today use the von Neumann model

14

- A: No.
- Qualified Answer: No. But, it has been the dominant way
 - i.e., the dominant paradigm for computing
 - for N decades

The Dataflow Execution Model of a Computer

The Dataflow Model (of a Computer)

- **Von Neumann model:** An instruction is fetched and executed in **control flow order**
 - ❑ As specified by the **program counter (instruction pointer)**
 - ❑ Sequential unless explicit control flow instruction

- **Dataflow model:** An instruction is fetched and executed in **data flow order**
 - ❑ i.e., when its operands are ready
 - ❑ i.e., there is **no program counter (instruction pointer)**
 - ❑ Instruction ordering specified by data flow dependence
 - Each instruction specifies “who” should receive the result
 - An instruction can “fire” whenever all operands are received
 - ❑ Potentially many instructions can execute at the same time
 - Inherently more parallel

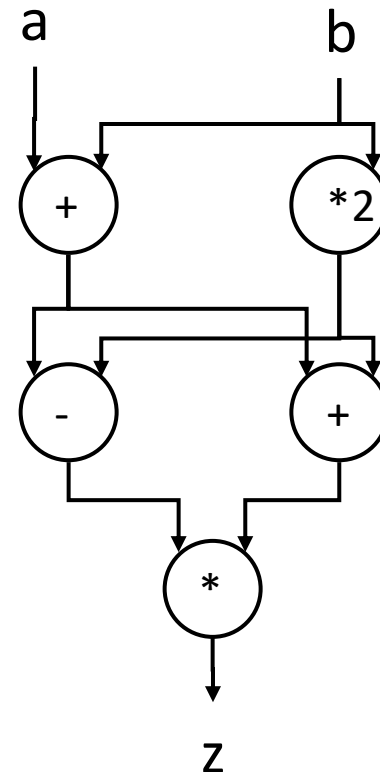
Von Neumann vs. Dataflow

- Consider a Von Neumann program
 - What is the significance of the program order?
 - What is the significance of the storage locations?

**$v = a + b;$
 $w = b * 2;$
 $x = v - w$
 $y = v + w$
 $z = x * y$**

Sequential

**a, b are the only inputs
 z is the only output**

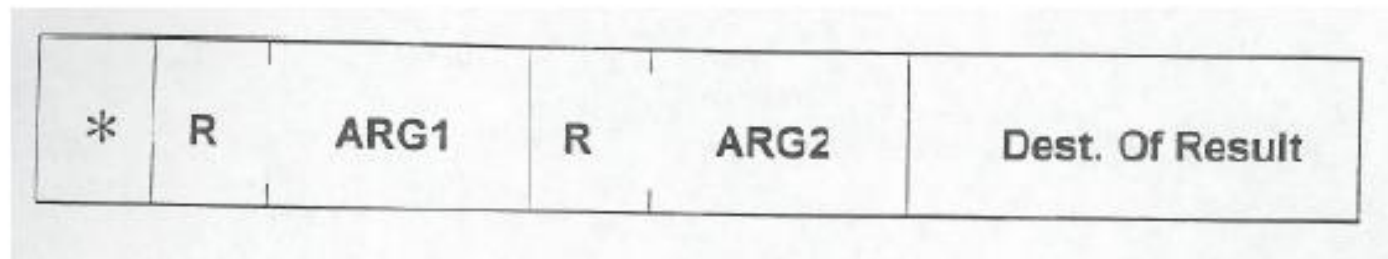
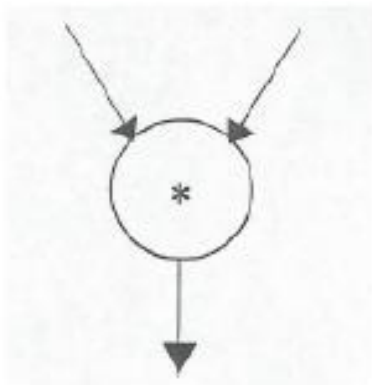


Dataflow

Which model is more natural to you as a programmer?

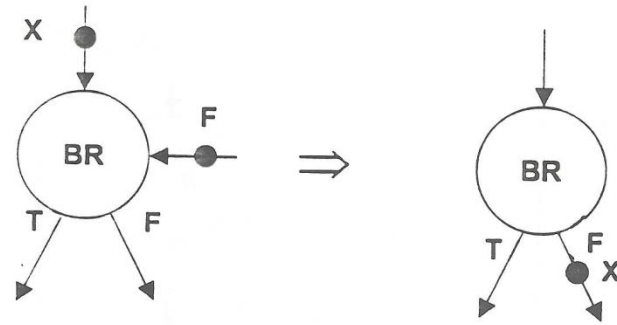
More on Dataflow

- In a dataflow machine, a program consists of dataflow nodes
 - A dataflow node fires (fetched and executed) when all its inputs are ready
 - i.e. when all inputs have tokens
- Dataflow node and its ISA representation

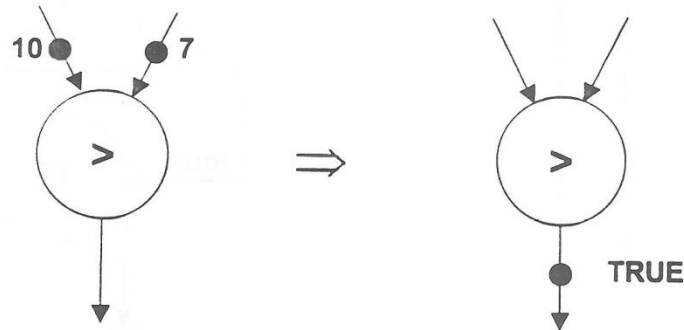


Example Dataflow Nodes

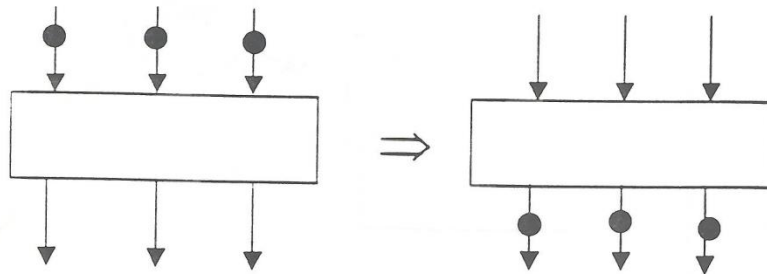
***Conditional**



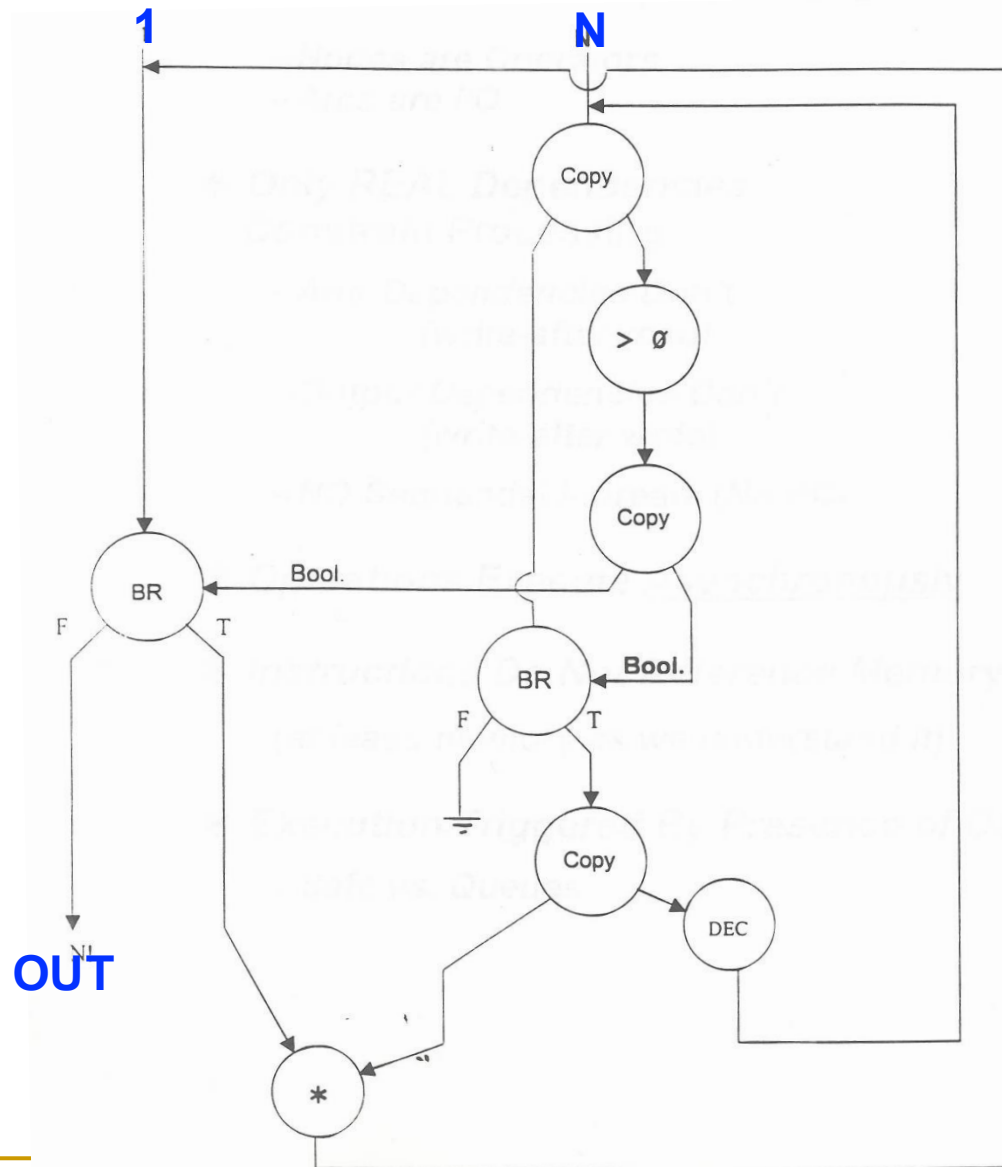
***Relational**



***Barrier Synchron**



A Simple Example Dataflow Program



**N is a
non-negative
integer**

**What is the
value of OUT?**

ISA-level Tradeoff: Program Counter

- Do we want a Program Counter (PC or IP) in the ISA?
 - Yes: Control-driven, sequential execution
 - An instruction is executed when the PC points to it
 - PC automatically changes sequentially (except for control flow instructions) → sequential
 - No: Data-driven, parallel execution
 - An instruction is executed when all its operand values are available → dataflow
- Tradeoffs: MANY high-level ones
 - Ease of programming (for average programmers)?
 - Ease of compilation?
 - Performance: Extraction of parallelism?
 - Hardware complexity?

ISA vs. Microarchitecture Level Tradeoff

- A similar tradeoff (control vs. data-driven execution) can be made at the microarchitecture level
- ISA: Specifies how the **programmer sees** the instructions to be executed
 - Programmer sees a sequential, control-flow execution order vs.
 - Programmer sees a dataflow execution order
- Microarchitecture: How the **underlying implementation actually executes** instructions
 - Microarchitecture can execute instructions in any order as long as it obeys the semantics specified by the ISA when making the instruction results visible to software
 - Programmer should see the order specified by the ISA

Let's Get Back to the von Neumann Model

- But, if you want to learn more about dataflow...
- Dennis and Misunas, "A preliminary architecture for a basic data-flow processor," ISCA 1974.
- Gurd et al., "The Manchester prototype dataflow computer," CACM 1985.
- A later lecture
- If you are really impatient:
 - <http://www.youtube.com/watch?v=D2uue7izU2c>
 - <http://www.ece.cmu.edu/~ece740/f13/lib/exe/fetch.php?media=onur-740-fall13-module5.2.1-dataflow-part1.ppt>

Lecture Video on Dataflow Architectures

The video content includes two slides titled "Loops and Function Calls Summary".

Slide 1 (Left): Shows a graph of a loop with a "tag" node and a "Send to Destination" node. The graph includes a "while" loop and a "do" block. The text below the graph reads: "Figure 11. Interface for a procedure call. On the left a call of procedure P whose graph is on the right. P has one parameter and one return value. The actual parameter receives a new tag and is sent to the input node of P and concurrently a token containing address A is sent to the output node. This SEND-TO-DESTINATION node transmits the other input token to a node of which the address is contained in the first token. The effect is that, when the return value of the procedure becomes available, the output node sends the result to node A, which returns the tag belonging to the calling expression."

Slide 2 (Right): Shows a graph of a loop with a "tag" node and a "Send to Destination" node. The graph includes a "while" loop and a "do" block. The text below the graph reads: "Figure 12. An implementation of a loop using tagged values. At the start of the loop a new tag area is allocated. Values belonging to consecutive iterations arrive consecutive tags within the area. The tag from before the loop is inserted on tokens that exit from the loop."

The chalkboard shows the following code and diagrams:

```
S + A[j] * B[j]
```

```
<f=0, 3, L, 0>
```

```
<f=0, 3, R, 1000>
```

The video player interface shows a progress bar at 42:27 / 1:25:00. Below the video, the title "Carnegie Mellon - Parallel Computer Architecture 2012-Onur Mutlu - Lec 22 - Dataflow I" is displayed, along with "3,627 views • Apr 21, 2013". The video is from the channel "Carnegie Mellon Computer Architecture" with "1.79K subscribers". The video has 24 likes and 0 comments. The video is shared and saved. The video is subscribed to.

The von Neumann Model

- All major *instruction set architectures* today use this model
 - x86, ARM, MIPS, SPARC, Alpha, POWER, RISC-V, ...
- Underneath (at the microarchitecture level), the execution model of almost all *implementations (or, microarchitectures)* is very different
 - Pipelined instruction execution: *Intel 80486 uarch*
 - Multiple instructions at a time: *Intel Pentium uarch*
 - Out-of-order execution: *Intel Pentium Pro uarch*
 - Separate instruction and data caches
- But, what happens underneath that is **not consistent** with the von Neumann model is **not exposed** to software
 - Difference between ISA and microarchitecture

What is Computer Architecture?

- **ISA+implementation definition:** The science and art of designing, selecting, and interconnecting hardware components and designing the hardware/software interface to create a computing system that meets functional, performance, energy consumption, cost, and other specific goals.
- **Traditional (ISA-only) definition:** “The term *architecture* is used here to describe the attributes of a system as seen by the programmer, i.e., the conceptual structure and functional behavior *as distinct from* the organization of the dataflow and controls, the logic design, and the physical implementation.”

Gene Amdahl, IBM Journal of R&D, April 1964

ISA vs. Microarchitecture

■ ISA

- ❑ Agreed upon interface between software and hardware
 - SW/compiler assumes, HW promises
- ❑ What the software writer needs to know to write and debug system/user programs

■ Microarchitecture

- ❑ Specific implementation of an ISA
- ❑ Not visible to the software

■ Microprocessor

- ❑ **ISA, uarch**, circuits
- ❑ “Architecture” = ISA + microarchitecture

Problem
Algorithm
Program
ISA
Microarchitecture
Circuits
Electrons

Microarchitecture

- A specific **implementation** of the ISA
- How do we implement the ISA?
 - We will discuss this for many lectures
- There can be many implementations of the same ISA
 - **MIPS** R2000, R3000, R4000, R6000, R8000, R10000, ...
 - **x86**: Intel 80486, Pentium, Pentium Pro, Pentium 4, Kaby Lake, Coffee Lake, Comet Lake, Ice Lake, Golden Cove, Sapphire Rapids, ..., AMD K5, K7, K9, Bulldozer, BobCat, Ryzen X, ...
 - **POWER** 4, 5, 6, 7, 8, 9, 10 (IBM), ..., **PowerPC** 604, 605, 620, ...
 - **ARM** Cortex-M*, ARM Cortex-A*, NVIDIA Denver, Apple A*, M1, ...
 - **Alpha** 21064, 21164, 21264, 21364, ...
 - **RISC-V** ...
 - ...

ISA vs. Microarchitecture

- What is part of ISA vs. Uarch?
 - ❑ Gas pedal: interface for “acceleration”
 - ❑ Internals of the engine: implement “acceleration”
- Implementation (uarch) can be various as long as it satisfies the specification (ISA)
 - ❑ Add instruction vs. Adder implementation
 - Bit serial, ripple carry, carry lookahead adders are all part of microarchitecture **(see H&H Chapter 5.2.1)**
 - ❑ x86 ISA has many implementations:
 - Intel 80486, Pentium, Pentium Pro, Pentium 4, Kaby Lake, Coffee Lake, Comet Lake, Ice Lake, Golden Cover, Sapphire Rapids, ..., AMD K5, K7, K9, Bulldozer, BobCat, Ryzen X, ...
- Microarchitecture usually changes faster than ISA
 - ❑ Few ISAs (x86, ARM, SPARC, MIPS, Alpha, RISC-V) but many uarchs
 - ❑ *Why?*



ISA: What Does It Specify?

■ Instructions

- ❑ Opcodes, Addressing Modes, Data Types
- ❑ Instruction Types and Formats
- ❑ Registers, Condition Codes

■ Memory

- ❑ Address space, Addressability, Alignment
- ❑ Virtual memory management

■ Call, Interrupt/Exception Handling

■ Access Control, Priority/Privilege

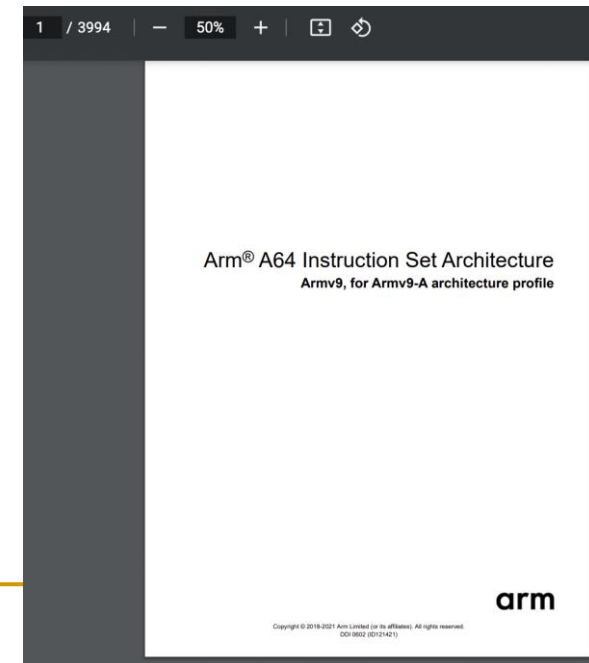
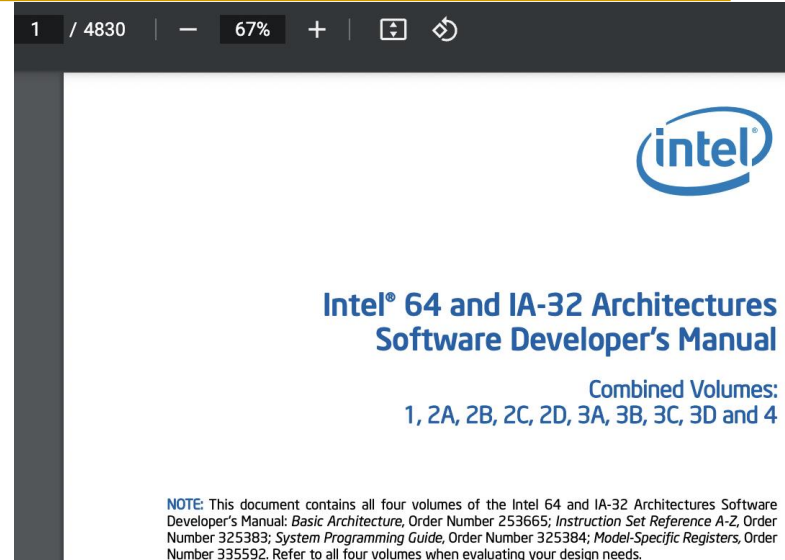
■ I/O: memory-mapped vs. instructions

■ Task/thread Management

■ Power & Thermal Management

■ Multithreading & Multiprocessor support

■ ...



ISAs Keep Getting Extended



ISAs Keep Getting Extended

1 / 5158



100%



Arm® A64 Instruction Set for A-profile architecture

ISA Manuals: Some Good Bedtime Reading

Combined Volume Set of Intel® 64 and IA-32 Architectures Software Developer's Manuals

Document	Description
Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4	This document contains the following: Volume 1: Describes the architecture and programming environment of processors supporting IA-32 and Intel® 64 architectures. Volume 2: Includes the full instruction set reference, A-Z. Describes the format of the instruction and provides reference pages for instructions. Volume 3: Includes the full system programming guide, parts 1, 2, 3, and 4. Describes the operating-system support environment of Intel® 64 and IA-32 architectures, including: memory management, protection, task management, interrupt and exception handling, multi-processor support, thermal and power management features, debugging, performance monitoring, system management mode, virtual machine extensions (VMX) instructions, Intel® Virtualization Technology (Intel® VT), and Intel® Software Guard Extensions (Intel® SGX). NOTE: Performance monitoring events can be found here: https://perfmon-events.intel.com/ Volume 4: Describes the model-specific registers of processors supporting IA-32 and Intel® 64 architectures.
Intel® 64 and IA-32 Architectures Software Developer's Manual Documentation Changes	Describes bug fixes made to the Intel® 64 and IA-32 architectures software developer's manual between versions. NOTE: This change document applies to all Intel® 64 and IA-32 architectures software developer's manual sets (combined volume set, 4 volume set, and 10 volume set).

ISA Manuals: Some Good Bedtime Reading



Specifications

The RISC-V instruction set architecture (ISA) and related specifications are developed, ratified and maintained by [RISC-V International contributing members](#) within the RISC-V International [Technical Working Groups](#). Work on the specification is [performed on GitHub](#), and the GitHub [issue mechanism](#) can be used to provide input into the specification.

If you would like more information on becoming a member, please see the [membership page](#).

ISA Specification

The specifications shown below represent the current, ratified releases. Work is being done on [GitHub](#).

- Volume 1, Unprivileged Spec v. 20191213 [\[PDF\]](#)
- Volume 2, Privileged Spec v. 20211203 [\[PDF\]](#)
- Recently ratified, but not yet integrated, [extension specifications](#)

Debug Specification

This is the currently ratified specification:

- External Debug Support v. 0.13.2 [\[PDF\]](#) [\[GitHub\]](#)

This is the current stable draft:

- External Debug Support v. 1.0.0-STABLE [\[PDF\]](#)

Trace Specification

The processor trace specification was **approved** on March 20, 2020.

- Trace Specification v. 1.0 [\[PDF\]](#) [\[GitHub\]](#)

Compatibility Test Framework

The RISC-V Architectural Compatibility Test Framework Version 2 is now available. This framework compares arbitrary models against a reference signature, and currently covers RV[32|64]IMC unprivileged specifications only. Tests for the not-yet-ratified Crypto Scalar extension and RV32EMC extensions are also available.

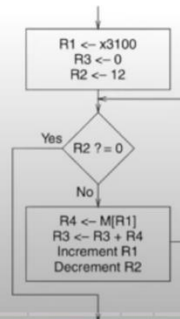
Work on Version 3.0 framework (RISCOF) is

Lecture 9b: Assembly Programming

A Program for Adding Integers in LC-3

- We use conditional branch instructions to create a loop

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	LEA	1	1	0	0	0	1	0	x00FF	1	1	1	1	1	1	1	R1 = PC + 0x00FF = 3100 // load address
x3001	AND	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 = 0 // reset register
x3002	AND	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 = 0 // reset register
x3003	ADD	0	0	1	0	1	0	0	1	0	1	0	1	1	0	0	R2 = R2 + 12 // initialize counter
x3004	BR	0	0	0	0	2	0	5	0	0	0	0	0	0	1	0	BRz (PC + 5) = BRz 0x300A // check condition
x3005	LDR	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 = M[R1 + 0] // load value
x3006	ADD	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	R3 = R3 + R4 // accumulate
x3007	ADD	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 = R1 + 1 // increment address
x3008	ADD	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 = R2 - 1 // decrement counter
x3009	BR	0	0	0	n	z	p	6	1	1	1	1	1	0	1	0	BRnzp (PC + 6) = BRnzp 0x3004 // jump



Digital Design & Computer Architecture - Lecture 10b: Assembly Programming (Spring 2022)



Onur Mutlu Lectures
31.8K subscribers

Analytics

Edit video

46



Share

Download

Clip

Save



2K views 11 months ago Livestream - Digital Design and Computer Architecture - ETH Zürich (Spring 2022)

Digital Design and Computer Architecture, ETH Zürich, Spring 2022 (<https://safari.ethz.ch/digitaltechnik...>)

Lecture 9b: Assembly Programming

Function Calls: Code Example

High-level code

```
int main()
{
    int y;
    ...
    // 4 arguments
    y = diffofsums(2, 3, 4, 5);
    ...
}

int diffofsums(int f, int g,
               int h, int i)
{
    int result;
    result = (f + g) - (h + i);
    // return value
    return result;
}
```

MIPS assembly

```
# $s0 = y
main:
...
addi $a0, $0, 2    # argument 0 = 2
addi $a1, $0, 3    # argument 1 = 3
addi $a2, $0, 4    # argument 2 = 4
addi $a3, $0, 5    # argument 3 = 5
jal  diffofsums    # call procedure
add  $s0, $v0, $0   # y = returned value
...

# $s0 = result
diffofsums:
add  $t0, $a0, $a1  # $t0 = f + g
add  $t1, $a2, $a3  # $t1 = h + i
sub  $s0, $t0, $t1  # result=(f + g) - (h + i)
add  $v0, $s0, $0   # put return value in $v0
jr   $ra           # return to caller
```

Argument values (red arrow pointing to \$a0-\$a3)

Return value (blue arrow pointing to \$v0)

Return address (green arrow pointing to \$ra)

Digital Design & Computer Architecture - Lecture 10b: Assembly Programming (Spring 2022)



Onur Mutlu Lectures
31.8K subscribers

Analytics

Edit video

46



Share

Download

Clip

Save



2,069 views Premiered Mar 28, 2022 Livestream - Digital Design and Computer Architecture - ETH Zürich (Spring 2022)
Digital Design and Computer Architecture, ETH Zürich, Spring 2022 (<https://safari.ethz.ch/digitaltechnik...>)